# Introducing RStudio

## Hands-On Learning and Career Readiness in the Social, Behavioral, and Educational Sciences



Njesa Totty

# Introducing RStudio

## Hands-On Learning and Career Readiness in the Social, Behavioral, and Educational Sciences

Njesa Totty

Assistant Professor

Framingham State University

# Copyright

Copyright © 2025. First Edition. Updated: August 31st, 2025.

Introducing RStudio: Hands-On Learning and Career Readiness in the Social, Behavioral, and Educational Sciences © 2025 by Njesa Totty is licensed under CC BY-NC 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc/4.0/.

This manual and its companion files may be downloaded from the author's GitHub page.

The creation of this manual was supported by the CA-ROTEL Grant. More information on the project may be found at https://carotelproject.org/.

This work was primarily human-created. AI-assisted tools were used during development for tasks including drafting and revising text, generating and refining ideas, developing examples and analyses, creating or adapting images, and organizing and formatting content. AI-generated or AI-assisted material was reviewed, revised, and approved by the author.

# Contents


**1 Preface** **7**

1.1 Use of GSS Data . . . 7
1.2 Career Readiness and NACE Competencies . . . 8
1.3 Use of AI in Development . . . 8
1.4 Using AI as a Learning Tool . . . 8
1.5 How to Use This Manual . . . 9

**2 Navigating the RStudio IDE** **10**

2.1 The RStudio Interface . . . 10
2.2 Installing and Loading Packages . . . 10
2.3 Console vs. Source . . . 11
2.4 Common Beginner Mistakes and How to Fix Them . . . 11
2.5 Summary . . . 11
2.6 Career Readiness Connection . . . 12

**3 Introduction to the Tidyverse and Core Data Skills** **13**

3.1 Why Use the Tidyverse? . . . 13
3.2 Core Packages . . . 13
3.3 Installing and Loading the Tidyverse . . . 13
3.4 A Note on Tibbles . . . 14
3.5 The Pipe Operator (`|>`) . . . 14
3.6 The Five Core `dplyr` Verbs . . . 15
3.7 Working with Text: `stringr` Functions . . . 19
3.8 Summary . . . 21
3.9 Career Readiness Connection . . . 21

**4 Importing and Cleaning Data** **23**

4.1 Inspecting Data . . . 23
4.2 Handling Missing Data . . . 24
4.3 Reshaping Data . . . 24
4.4 Recoding Values . . . 25
4.5 Career Readiness Connection . . . 26
4.6 Summary . . . 26
4.7 Career Readiness Connection . . . 27

**5 Visualizing and Summarizing Data** **28**
5.1 Histograms: Distributions of Numerical Variables . . . 28
5.2 Line Graphs: Trends Over Time . . . 30
5.3 Scatterplots: Relationships Between Two Numerical Variables . . . 31
5.4 Boxplots: Comparing Numerical Distributions Across Groups . . . 32
5.5 Bar Charts: Summarizing Categorical Variables . . . 33
5.6 Bar Charts with Group Comparisons . . . 34
5.7 Faceting: Small Multiples for Subgroup Comparisons . . . 35
5.8 Summary . . . 36
5.9 Career Readiness Connection . . . 37

**6 Methods for the Mean** **38**
6.1 One-Sample $t$-Test for Mean . . . 38
6.2 Paired $t$-Test for Mean Difference . . . 40
6.3 Two-Sample $t$-Test for Difference in Means . . . 43
6.4 ANOVA and Pairwise $t$-tests . . . 46
6.5 Summary . . . 49
6.6 Career Readiness Connection . . . 49

**7 Methods for the Proportion** **50**
7.1 One-Sample $z$-Test for Proportion . . . 50
7.2 Two-Sample $z$-Test for Difference in Proportions . . . 51
7.3 Sign Test for Median . . . 53
7.4 Summary . . . 56
7.5 Career Readiness Connection . . . 56

**8 Regression Methods** **57**
8.1 Simple Linear Regression . . . 57
8.2 Multiple Linear Regression . . . 61
8.3 Summary . . . 66
8.4 Career Readiness Connection . . . 67

**9 Practice Problems** **68**
9.1 RStudio . . . 68
9.2 Tidyverse . . . 68
9.3 Cleaning . . . 68
9.4 Data Visualization and Summarization . . . 69
9.5 Methods for the Mean . . . 69
9.6 Methods for the Proportion . . . 70
9.7 Regression Methods . . . 70

**10 Appendix A: Shorter GSS Dataset Codebook** **72**

**11 Appendix B: Transitioning from Python to R Using AI** **73**

11.1 Why Use AI to Bridge R and Python? . . . . . 73

11.2 How to Prompt AI for Code Translation . . . . . 75

11.3 Translating Core Data Tasks . . . . . 79

11.4 Translating Visualizations . . . . . 82

11.5 Becoming an Independent Learner with AI . . . . . 87

11.6 Conclusion . . . . . 89

# 1 Preface

Data analysis is more than a set of formulaic procedures—it is a method for understanding the world around us. This lab manual was developed to help students begin working in RStudio so that they can analyze data to answer authentic questions in the social, behavioral, and educational sciences. By working directly with real datasets and using professional analytical tools, students will gain practical skills that bridge the gap between classroom learning and workplace application.

The activities in this manual align with the key topics of exploratory data analysis, inference, and prediction that are often taught in introductory statistics courses. Each activity invites students to use their skills in meaningful contexts, strengthening both data literacy and problem-solving abilities.

The activities are organized around questions rather than procedures. Instead of learning a statistical method first and then finding a reason to use it, students begin with a question that could arise in research, public discussion, or professional work. They explore the data, identify an appropriate method, carry out the analysis, and interpret what the results mean in context. This approach emphasizes statistical thinking as a process of asking and answering questions with evidence.

A distinctive feature of this manual is its integration of **RStudio** as the primary computational platform. RStudio provides a modern, open-source environment for data analysis, widely used in research and professional settings. Through repeated hands-on practice, students will learn to write and adapt R code, interpret outputs, create publication-quality graphics, and present results in ways that meet professional standards.

## 1.1 Use of GSS Data

To ground activities in relevant, real-world examples, this manual draws extensively from the General Social Survey (GSS) for years 1972–2024. The GSS contains a rich array of information on U.S. society, making it particularly valuable for students in the social, behavioral, and educational sciences. A subset of the data is available at the author's GitHub page, and its codebook is in the appendix. The full dataset for years 1972-2024 is available from the GSS Data Explorer, which instructors may use to create alternative subsets for classroom use.

The GSS is designed to represent adults ages 18 and older living in households in the United States. It does not represent all U.S. adults: residents of institutions and other group quarters are outside the target population, and, prior to 2006, the survey was limited to adults able to complete the interview in English.

Beginning in 2006, the GSS expanded its target population to include adults able to complete the interview in either English or Spanish. Thus, results from the GSS should be understood as estimates about the GSS target population rather than about every person residing in the United States. This distinction is particularly important when comparing results across the 1972–2024 period, because the population eligible for the survey changed in 2006.

The GSS is a probability sample, and its sampling procedures are designed to produce nationally representative estimates when the appropriate survey weights are used. There are, however, important qualifications to making statistical inferences from the GSS, especially in an introductory classroom setting. The GSS does not constitute a simple random sample: respondents are selected through a complex, clustered, and stratified sampling design, and the survey has experienced substantial nonresponse and changes in sampling and data-collection procedures over time. The survey provides weights to account for unequal probabilities of selection and nonresponse, as well as post-stratification to population benchmarks. Moreover, the clustering and stratification of observations affect the precision of estimates, so standard errors calculated under simple-random-sampling assumptions can be too small.

For these reasons, analyses that are intended to make formal population inferences should use the appropriate GSS weight and, where possible, account for the survey's strata and primary sampling units rather than treating the observations as an ordinary random sample. At the same time, these limitations do

not make the GSS unsuitable for classroom use. The data are especially useful for teaching students how empirical patterns are found, described, visualized, and interpreted in a large national survey. Instructors can simply make the inferential distinction explicit: an unweighted analysis of the classroom dataset demonstrates how to analyze survey data and describe patterns in the observed sample; a weighted, design-aware analysis is needed when the goal is to estimate characteristics of the GSS target population or conduct formal hypothesis tests about that population. This distinction also provides an opportunity to teach an important lesson about survey research: a dataset can be carefully designed to support population inference while still requiring analysts to understand sampling, weighting, nonresponse, and variance estimation before making inferential claims.

## 1.2 Career Readiness and NACE Competencies

In addition to developing data analysis skills, this manual is designed to help students connect their academic work to professional competencies valued in the workplace. The National Association of Colleges and Employers (NACE) identifies eight core career readiness competencies:

1. Career and Self-Development
2. Communication
3. Critical Thinking
4. Equity and Inclusion
5. Leadership
6. Professionalism
7. Teamwork
8. Technology

Each activity in this manual intentionally builds one or more of these competencies. For example, students may develop *Critical Thinking* by interpreting `ggplot` outputs to guide evidence-based decisions, or *Technology* skills by automating data cleaning processes in R. By engaging with these competencies throughout the manual, students can see how the skills they build translate into tangible career preparation.

## 1.3 Use of AI in Development

The creation of this manual incorporated AI-assisted tools for formatting, content organization, and resource curation. This reflects a commitment to leveraging emerging technologies in educational publishing, while ensuring that all instructional content has been carefully reviewed and aligned with learning objectives.

## 1.4 Using AI as a Learning Tool

AI is increasingly part of the tools used in research, education, and professional data analysis. This manual treats AI as a tool for learning and problem-solving, not as a replacement for learning R or understanding statistics.

Throughout the manual, you will encounter prompts that can be used with an AI tool to study concepts, troubleshoot R code, compare analytical approaches, and explore how statistical skills are used in different

careers. The goal is to use AI actively: ask it to explain, question its suggestions, test its code, and verify its claims rather than simply copying its answers.

A useful rule is: *use AI to help you think, not to do the thinking for you.* AI can produce incorrect code, misleading explanations, or results that appear reasonable but do not answer the question you asked. You are responsible for checking whether an AI-generated response is correct and appropriate for the data and research question.

## 1.5 How to Use This Manual

Students are encouraged to complete activities sequentially, as later exercises build on skills developed earlier. Instructors may adapt the activities to fit the structure of their courses or the specific needs of their students. Supplementary files are available on the author's GitHub page for easy access and customization.

By the end of this manual, students will not only be able to apply the methods of data visualization and summarization using RStudio but also recognize the value of these skills in a variety of career paths across the social, behavioral, and educational sciences.

# 2 Navigating the RStudio IDE

In this chapter, you will learn to navigate RStudio, install and load packages, and avoid common beginner pitfalls. By the end, you will be able to open a new R script, write and run R code, install the packages you need, and troubleshoot common errors. At this point, you should have RStudio downloaded and opened alongside this lab manual. Alternatively, you may create a Posit Cloud account and begin a new RStudio Project in the cloud.

## 2.1 The RStudio Interface

RStudio is divided into four panes:

- **Source (Top Left)**: Where you write and save R scripts. To open a new script, press Ctrl+Shift+N (Windows) or Cmd+Shift+N (Mac)
- **Console (Bottom Left)**: Where you run code for immediate execution.
- **Environment/History (Top Right)**: Lists objects and variables in memory, plus a history of commands.
- **Files/Plots/Packages/Help/Viewer/Presentation (Bottom Right)**: Used to browse files, view plots, documents, or presentations, manage packages, and access help files.

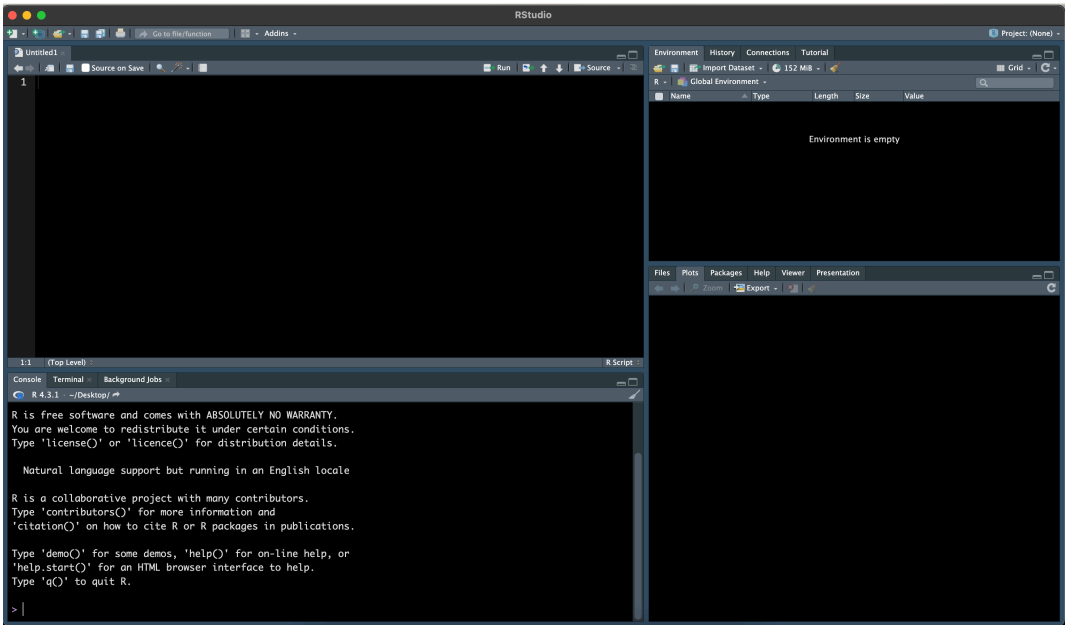



Figure 1: The four-pane layout of RStudio.

## 2.2 Installing and Loading Packages

R's core functions are extended by packages. We will use `tidyverse` frequently, so it is a good idea to install it now. Type the following code into your R script and use Ctrl+R (Windows) or Cmd+R (Mac) to run each line.

- Install once: `install.packages("tidyverse")`
- Load in each session: `library(tidyverse)`

Installing a package is like screwing in a light bulb. Loading a package is like turning on the light switch. You only need to install the package (light bulb) once, but you need to load the package (turn on the light) each time you open RStudio (enter the room).

## 2.3 Console vs. Source

The console is for one-time commands; the source is for writing and saving code.

- Try typing `2 + 2` in both places.
- You can save your R script with `Ctrl+S` (Windows) or `Cmd+S` (Mac).
- You can repopulate the console with the previous line of code using the up arrow
- Code typed into the Console is not saved!

## 2.4 Common Beginner Mistakes and How to Fix Them

Forgetting quotes around strings: `paste(apple)` causes an error; use `"apple"` for strings.

Running code in the wrong pane: If nothing happens, check if you typed into the source but forgot to press `Ctrl+Enter` (Windows) or `Cmd+Enter` (Mac).

Forgetting to load a package: Restarting R clears loaded packages, and their functions will not be found; run `library(package_name)` again.

Overwriting variables: Accidentally reusing names (e.g., `mean <- 5`) will hide the original function. Restart R to reset.

Misplaced working directory: Errors like “file not found” often mean R is looking in the wrong folder. Check with `getwd()`.

Knowing these pitfalls can save hours of frustration. Learning to code in R takes time and practice, so be prepared for some frustration and determine now to continue persevering despite this.

## 2.5 Summary

You now have the essential navigation skills to work effectively in RStudio. The more time you spend working in RStudio, the more comfortable you will become using these and other skills. There is a wealth of RStudio documentation online. It is always a good idea to search for help online and bookmark any useful tips for future reference.

## 2.6 Career Readiness Connection

The skills in this chapter support several career readiness competencies:

- **Technology**: Learning to work in a professional data-analysis environment.
- **Career and Self-Development**: Learning to find documentation and troubleshoot problems independently.
- **Professionalism**: Developing habits for organizing, saving, and documenting analytical work.

These skills may seem basic at first, but knowing how to navigate a technical environment, locate reliable documentation, and troubleshoot errors are transferable skills used across research and data-focused careers.

### 2.6.1 Using AI to Study and Apply This Chapter

Try the following prompts with an AI tool. Read the response critically and verify any R code before using it.

1. **Study the concepts:**
   `Explain the difference between the RStudio Source pane and Console as if I am a first-time R user. Give me two examples of when I should use each one.`
2. **Practice RStudio skills:**
   `Give me three short RStudio troubleshooting scenarios involving the Console, Source pane, packages, or the working directory. Do not give me the answers until I attempt them.`
3. **Debug an error:**
   `I received this error in R: [PASTE ERROR HERE]. Explain what the error means, identify the likely cause, and give me questions I can ask myself before showing me a solution.`
4. **Evaluate an AI answer:**
   `Explain why restarting R can sometimes solve a problem but does not necessarily fix the underlying issue. Give me an example of a problem that restarting R would not solve.`
5. **Connect to a career:**
   `Choose a career in [MY FIELD] that uses data. Explain how skills such as working with an IDE, reading documentation, troubleshooting errors, and organizing code are used in that career. Identify the claims I should verify with a reliable source.`

# 3 Introduction to the Tidyverse and Core Data Skills

The `tidyverse` is a collection of R packages designed to make data science easier, more consistent, and more readable. Think of it as a "toolbox" filled with tools that all work in a similar way, so once you learn one, the others feel familiar. We will use the `tidyverse` throughout this manual for data wrangling, visualization, and analysis.

While R has many built-in, or **base** `R`, functions, the tidyverse provides a consistent "grammar" for data analysis that's easier to learn for beginners and widely used by professionals in research, government, and industry.

## 3.1 Why Use the Tidyverse?

- **Consistency** – The same style and function names appear across different tasks.
- **Readability** – Code often reads like a sentence describing the analysis.
- **Efficiency** – Complex tasks can often be completed with fewer lines of code.
- **Career relevance** – Tidyverse skills are in demand for data roles across the social, behavioral, and educational sciences.

## 3.2 Core Packages

When you load the tidyverse, you get several packages at once:

| Package | Purpose |
|---|---|
| `ggplot2` | Create high-quality visualizations |
| `dplyr` | Manipulate and summarize data (filter rows, select columns, group, arrange) |
| `tidyr` | Reshape and clean data |
| `readr` | Import data from CSV and text files |
| `stringr` | Work with text data (strings) |
| `forcats` | Manage categorical variables (factors) |

## 3.3 Installing and Loading the Tidyverse

You only need to install the `tidyverse` **once** on your computer. The code is included below in case you have not already done so.

```
install.packages("tidyverse")
```

You must **load** the `tidyverse` each time you open RStudio:

```
library(tidyverse)
```

## 3.4 A Note on Tibbles

When you work with `tidyverse` functions, you will often see data printed in a slightly different format called a **tibble**. Tibbles are modern versions of data frames:

- They show only the first few rows and columns by default.
- They display column data types under the names.
- They never change variable names or types without your permission.

## 3.5 The Pipe Operator (`|>`)

A key feature of tidyverse workflow is the **pipe** operator, `|>`. It passes the result of one function into the next.

Without the pipe:

```
summarise(group_by(txhousing, year), avg_sales = mean(sales, na.rm = TRUE))
```

```
## # A tibble: 16 x 2
##     year avg_sales
##    <int>     <dbl>
##  1  2000      478.
##  2  2001      496.
##  3  2002      532.
##  4  2003      539.
##  5  2004      577.
##  6  2005      636.
##  7  2006      665.
##  8  2007      621.
##  9  2008      520.
## 10  2009      469.
## 11  2010      440.
## 12  2011      447.
## 13  2012      520.
## 14  2013      608.
## 15  2014      626.
## 16  2015      659.
```

With the pipe:

```
txhousing |>
  group_by(year) |>
  summarise(avg_sales = mean(sales, na.rm = TRUE))
```

```
## # A tibble: 16 x 2
##     year avg_sales
##    <int>     <dbl>
##  1  2000      478.
##  2  2001      496.
##  3  2002      532.
```

```
##  4  2003      539.
##  5  2004      577.
##  6  2005      636.
##  7  2006      665.
##  8  2007      621.
##  9  2008      520.
## 10  2009      469.
## 11  2010      440.
## 12  2011      447.
## 13  2012      520.
## 14  2013      608.
## 15  2014      626.
## 16  2015      659.
```

Think of it as saying:

1. Start with `txhousing`.
2. Group by `year`.
3. Summarize with the average number of sales.

We'll explain what `summarise` and `group_by` do later but notice that both chunks of code produce the same result. However, code written with the pipe is easier to read.

## 3.6 The Five Core `dplyr` Verbs

In the following examples, we will use the **General Social Survey** dataset. Read in the dataset from your computer and save it as `gssdat`. The dataset should be saved in the same location on your computer as the R script that you are working in.

```
# read from a downloaded file in your working directory
gssdat <- read.csv("gssdat.csv")
```

### 3.6.1 `select()` — Choose the variables you need

Datasets can be overwhelming. If you check the dimensions of the GSS data with `dim(gssdat)`, you will see that it is a large dataset. Selecting specific variables keeps your workspace and analysis focused. For example, you can select the variables `year`, `age`, `degree`, and `income` using the code below. We use `slice_sample(n = 10)` to return only a random sample of ten rows rather than the entire dataset. We will use this throughout, and it should be used before knitting to avoid generating cumbersome output.

```
gssdat |>
  select(year, age, degree, income) |>
  slice_sample(n = 10)
```

```
##    year age                degree income
## 1  1987  43           high school     12
## 2  2024  42              graduate     12
## 3  2004  46           high school     11
## 4  1994  53           high school     12
## 5  1977  23                  <NA>      6
```

```
## 6  2008  44                 graduate     12
## 7  1996  22              high school      7
## 8  1998  25              high school      5
## 9  2018  37 associate/junior college     12
## 10 2010  47               bachelor's     12
```

---

### 3.6.2 `filter()` — Keep only the rows you want

Social science researchers often focus on specific subgroups (e.g., only people of a certain age, education level, or survey year). Filtering lets us zoom in. For example, we may want to study the `age`, `degree`, and `income` of respondents from 2022 who have a bachelor's degree or higher.

```
gssdat |>
  select(year, age, degree, income) |>
  filter(year == 2022, degree %in% c("bachelor's", "graduate")) |>
  slice_sample(n = 10)
```

```
##    year age     degree income
## 1  2022  69   graduate     12
## 2  2022  74   graduate     12
## 3  2022  74 bachelor's     12
## 4  2022  74 bachelor's      1
## 5  2022  79 bachelor's     12
## 6  2022  74   graduate     12
## 7  2022  52 bachelor's     12
## 8  2022  55 bachelor's     12
## 9  2022  65 bachelor's     12
## 10 2022  38 bachelor's     12
```

The `%in%` operator determines whether elements of the vector on the left-hand side are present in the right-hand side vector. There are many ways to use this operator. A few examples are given below along with the output that they return. The length of the resulting logical vector will be the same as that of the left-hand side vector.

```
# length one vector
5 %in% c(1, 3, 5, 7, 9)
```

```
## [1] TRUE
```

```
# length three vector
c(1, 3, 5) %in% c(1, 2, 4, 6, 3)
```

```
## [1]  TRUE  TRUE FALSE
```

```
# toy dataset
toydat <- data.frame(letter = c("A", "B", "C", "D", "E", "F", "I", "A"))

# length two vector against variable
c("A", "F") %in% toydat$letter
```

```
## [1] TRUE TRUE
```

```
# variable against length two vector
toydat$letter %in% c("A", "F")
```

```
## [1]  TRUE FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE
```

---

### 3.6.3 `mutate()` — Create new variables

Sometimes the variable you need is not readily available in the dataset, and you must create it from other variables in the dataset. For example, we can create a new variable for whether a respondent is considered a "young adult" (18–30) or not.

```
gssdat |>
  mutate(young_adult = if_else(age >= 18 & age <= 30, "Yes", "No")) |>
  select(year, young_adult, degree, income) |>
  slice_sample(n = 10)
```

```
##    year young_adult                degree income
## 1  1990          No less than high school     12
## 2  2000          No             bachelor's     12
## 3  2012          No           high school     10
## 4  1983          No           high school     10
## 5  2022          No           high school     NA
## 6  1987          No           high school     12
## 7  1990          No less than high school      9
## 8  2018          No             bachelor's     12
## 9  1978          No less than high school      2
## 10 2018          No less than high school     NA
```

---

### 3.6.4 `summarise()` — Condense data into summaries

Summarizing helps researchers describe trends, central tendencies, and group differences. For example, we can find the average age of respondents by highest degree. Setting `na.rm = TRUE` removes missing values before R calculates the mean. Missing values are commonly found in all types of data, including survey data where respondents may choose not to answer a question.

```
gssdat |>
  group_by(degree) |>
  summarise(avg_age = mean(age, na.rm = TRUE))
```

```
## # A tibble: 6 x 2
##   degree                   avg_age
##   <chr>                      <dbl>
## 1 associate/junior college    44.0
```

```
## 2 bachelor's                  45.2
## 3 graduate                    49.9
## 4 high school                 44.8
## 5 less than high school       52.5
## 6 <NA>                        47.4
```

---

### 3.6.5 `arrange()` — Order the rows

Sorting results can make patterns easier to see and tables easier to interpret. For example, we can arrange degrees by average income bracket from lowest to highest.

```
gssdat |>
  group_by(degree) |>
  summarise(avg_income = mean(income, na.rm = TRUE)) |>
  arrange(avg_income)
```

```
## # A tibble: 6 x 2
##   degree                   avg_income
##   <chr>                         <dbl>
## 1 less than high school          8.08
## 2 <NA>                           8.45
## 3 high school                   10.3
## 4 associate/junior college      11.1
## 5 bachelor's                    11.3
## 6 graduate                      11.6
```

The GSS `income` variable uses numeric codes for income brackets. These codes should be treated as categories rather than as continuous dollar amounts. The corresponding labeled variable `incomecat` makes those categories easier to interpret.

```
gssdat |>
  count(incomecat)
```

```
##             incomecat     n
## 1    $1,000 to $2,999  1460
## 2  $10,000 to $14,999  7025
## 3  $15,000 to $19,999  5404
## 4  $20,000 to $24,999  5667
## 5     $25,000 or more 37051
## 6    $3,000 to $3,999  1335
## 7    $4,000 to $4,999  1207
## 8    $5,000 to $5,999  1339
## 9    $6,000 to $6,999  1263
## 10   $7,000 to $7,999  1331
## 11   $8,000 to $9,999  2319
## 12       under $1,000   969
## 13               <NA>  9329
```

Sorting results can make patterns easier to see and tables easier to interpret. For example, we can arrange education groups by their average age from youngest to oldest.

```
gssdat |>
  group_by(degree) |>
  summarise(avg_age = mean(age, na.rm = TRUE)) |>
  arrange(avg_age)
```

```
## # A tibble: 6 x 2
##   degree                   avg_age
##   <chr>                      <dbl>
## 1 associate/junior college    44.0
## 2 high school                 44.8
## 3 bachelor's                  45.2
## 4 <NA>                        47.4
## 5 graduate                    49.9
## 6 less than high school       52.5
```

To reverse the order, use `desc()`:

```
gssdat |>
  group_by(degree) |>
  summarise(avg_age = mean(age, na.rm = TRUE)) |>
  arrange(desc(avg_age))
```

```
## # A tibble: 6 x 2
##   degree                   avg_age
##   <chr>                      <dbl>
## 1 less than high school       52.5
## 2 graduate                    49.9
## 3 <NA>                        47.4
## 4 bachelor's                  45.2
## 5 high school                 44.8
## 6 associate/junior college    44.0
```

---

## 3.7 Working with Text: `stringr` Functions

Data cleaning is not limited to numerical variables. Survey datasets often contain text such as occupation titles, and researchers may need to search, identify, or standardize those responses before analyzing them.

### 3.7.1 `str_detect()` — Find patterns in text

Many survey variables are stored as text (e.g., job titles, open-ended responses). Pattern matching helps you find specific entries. For example, we can find respondents whose job titles contain “teacher”, “nurse”, or “emergency”.

```
gssdat |>
  filter(str_detect(occ10, regex("teacher|nurse|emergency"))) |>
  select(age, incomecat, occ10) |>
  slice_sample(n = 10)
```

```
##    age          incomecat                                        occ10
## 1   41    $25,000 or more                                registered nurses
## 2   37    $25,000 or more             elementary and middle school teachers
## 3   41    $25,000 or more                                registered nurses
## 4   77   $8,000 to $9,999             elementary and middle school teachers
## 5   47    $25,000 or more                                registered nurses
## 6   72    $25,000 or more                                registered nurses
## 7   80 $15,000 to $19,999 licensed practical and licensed vocational nurses
## 8   29 $10,000 to $14,999 licensed practical and licensed vocational nurses
## 9   29    $25,000 or more                          secondary school teachers
## 10  34    $25,000 or more                          secondary school teachers
```

In the above code, the bar (|) is interpreted as `or`; asking, '`Do any of these terms appear?" To filter based on the detection of a single string, we do not need to use`regex' as below.

```
gssdat |>
  filter(str_detect(occ10, "teacher")) |>
  select(age, incomecat, occ10) |>
  slice_sample(n = 10)
```

```
##    age          incomecat                                  occ10
## 1   29    $25,000 or more elementary and middle school teachers
## 2   42               <NA>             secondary school teachers
## 3   89    $25,000 or more elementary and middle school teachers
## 4   70    $25,000 or more                postsecondary teachers
## 5   38    $25,000 or more        other teachers and instructors
## 6   78    $25,000 or more elementary and middle school teachers
## 7   31    $25,000 or more            special education teachers
## 8   36    $25,000 or more                    teacher assistants
## 9   27 $10,000 to $14,999             secondary school teachers
## 10  35    $25,000 or more elementary and middle school teachers
```

---

### 3.7.2 `str_replace()` — Clean up or standardize text

Standardizing text responses can make them easier to analyze. Cleaning up strings with repetition can also make analysis easier. For example, we can shorten the word miscellaneous to misc for brevity in the occupation variable.

```
gssdat |>
  filter(str_detect(occ10, "miscellaneous")) |>
  mutate(occ10 = str_replace(occ10, "miscellaneous", "misc")) |>
  select(occ10)  |>
  slice_sample(n = 10)
```

```
##                                          occ10
## 1                      misc agricultural workers
## 2                  misc assemblers and fabricators
## 3                      misc agricultural workers
## 4           misc health technologists and technicians
```

```
## 5                      misc assemblers and fabricators
## 6                            misc agricultural workers
## 7                            misc agricultural workers
## 8                      misc assemblers and fabricators
## 9  misc life, physical, and social science technicians
## 10                     misc assemblers and fabricators
```

---

## 3.8 Summary

In this chapter, we learned how to use core `tidyverse` tools to work with data:

- `select()` chooses variables.
- `filter()` chooses observations.
- `mutate()` creates or changes variables.
- `summarise()` calculates summaries.
- `group_by()` produces summaries for groups.
- `arrange()` orders results.
- `str_detect()` and `str_replace()` help work with text.

These functions form the foundation for the data-analysis workflow used throughout the rest of the manual.

## 3.9 Career Readiness Connection

The skills in this chapter support several career readiness competencies:

- **Technology**: Using code-based tools to organize and manipulate data.
- **Critical Thinking**: Deciding which observations and variables are relevant to a question.
- **Communication**: Producing organized results that can be interpreted and shared with others.
- **Professionalism**: Developing reproducible workflows rather than manually editing data.

These are transferable skills used in research, business, government, education, healthcare, and other data-focused careers.

### 3.9.1 Using AI to Study and Apply This Chapter

Try the following prompts with an AI tool. Ask the AI to explain its reasoning rather than simply giving you code and test any code in RStudio.

1. **Study the concepts:**
   `Teach me the difference between select(), filter(), mutate(), summarise(), group_by(), and arrange() in R. Give me a small example of each and then quiz me without giving me the answers.`
2. **Practice R:**
   `Give me a small practice dataset and ask me three questions that require select(), filter(), mutate(), and summarise(). Do not provide the R code until I attempt each question.`

3. **Debug R code:**
   `Here is my R code: [PASTE CODE]. Explain what each step is doing, identify any errors, and suggest how I can debug it without simply rewriting the entire solution.`

4. **Evaluate an AI response:**
   `Give me an example of a plausible but incorrect use of group_by() and summarise() in R. Explain why the result could be misleading.`

5. **Connect to a career:**
   `How might data-wrangling skills using R and the tidyverse be used in [MY FIELD]? Give me three concrete examples of tasks a professional might perform and identify reliable sources I could use to verify them.`

# 4 Importing and Cleaning Data

In this section, you will learn how to import, inspect, and clean data using the **tidyverse**. These are the first steps in almost every data analysis project, and mastering them will help you prepare datasets for meaningful analysis.

## 4.1 Inspecting Data

We will continue using the **General Social Survey (GSS)** dataset introduced earlier. If you have not already loaded it, you can do so now. If you closed RStudio since the last section, reload the dataset before proceeding.

If you already have the `gssdat` dataset and `tidyverse` loaded from Section 1.2, you can skip this step. Otherwise, run the following code:

```
# load package
library(tidyverse)

# Load the GSS dataset from your working directory
gssdat <- read.csv("gssdat.csv")
```

Before cleaning data, it's important to understand what you have. Use these functions to explore the dataset:

```
# Number of rows and columns
dim(gssdat)
```

```
## [1] 75699    30
```

```
# First ten variable names
names(gssdat)
```

```
##  [1] "year"      "age"       "degree"    "income"    "occ10"     "incomecat"
##  [7] "trustfam"  "trustdoc"  "wrkstat"   "id"        "marital"   "agewed"
## [13] "conjudge"  "realrinc"  "hrs1"      "conpress"  "race"      "consci"
## [19] "fepol"     "happy"     "childs"    "health"    "realinc"   "coneduc"
## [25] "conlegis"  "sex"       "educ"      "paeduc"    "relig"     "res16"
```

```
# Summary of one variable
summary(gssdat$age)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##   18.00   32.00   44.00   46.72   60.00   89.00     870
```

These simple checks help you identify issues such as missing values, inconsistent variable names, or unusual data entries.

## 4.2 Handling Missing Data

Missing values are common in survey data. They are often represented by `NA` in R. We should not automatically remove every row containing a missing value. Instead, we should determine which variables are needed for the analysis and handle missing values appropriately for that analysis. For example, `na.rm = TRUE` can exclude missing values when calculating a summary statistic, while `filter(!is.na(variable))` can restrict an analysis to respondents with valid values for a particular variable. Below are some examples of how we can handle missing data:

```
# Count number of missing values in each column
gssdat |>
  is.na() |>
  colSums()
```

```
##      year       age    degree    income     occ10 incomecat  trustfam  trustdoc
##         0       870       199      9329      5337      9329     74321     74334
##   wrkstat        id   marital   agewed  conjudge  realrinc      hrs1  conpress
##        47         0        66     49156     26388     31490     32371     25474
##      race    consci     fepol     happy    childs    health   realinc   coneduc
##       172     27826     39108      4830       292     17251      7856     25135
##  conlegis       sex      educ    paeduc     relig     res16
##     25853       131       286     21781       477      1654
```

```
# How many rows do not have a missing income bracket
gssdat |>
  filter(!is.na(income)) |>
  nrow()
```

```
## [1] 66370
```

How missing data are handled can affect both the results and the conclusions of an analysis. Good data cleaning therefore requires understanding why observations are missing and considering how the chosen approach may affect the analysis.

## 4.3 Reshaping Data

Sometimes variables are stored in a “wide” format but need to be “long” for analysis. The pivot_longer() function from `tidyr` changes wide data to long.

Suppose that we want to compare responses to trust questions across types. Below we compare types of trusts between trusting one’s family versus one’s doctor.

```
# Example with two trust variables
gssdat |>
  select(trustfam, trustdoc) |>
  pivot_longer(cols = c(trustfam, trustdoc),
               names_to = "trust_type",
               values_to = "trust_level") |>
  group_by(trust_type, trust_level) |>
  summarize(count = n())
```

```
## # A tibble: 12 x 3
## # Groups:   trust_type [2]
##    trust_type trust_level   count
##    <chr>      <chr>         <int>
##  1 trustdoc   a great deal    400
##  2 trustdoc   completely      381
##  3 trustdoc   not at all       49
##  4 trustdoc   only a little    94
##  5 trustdoc   somewhat        441
##  6 trustdoc   <NA>          74334
##  7 trustfam   a great deal    242
##  8 trustfam   completely     1025
##  9 trustfam   not at all       17
## 10 trustfam   only a little    17
## 11 trustfam   somewhat         77
## 12 trustfam   <NA>          74321
```

We can also expand the data in one column based on another column using `pivot_wider()`. Suppose that we want to view the work status of respondents for surveys after 2020.

```
# widen workstatus by year
gssdat |>
  select(id, year, wrkstat) |>
  pivot_wider(names_from = year,
              values_from = wrkstat) |>
  select(`2021`, `2022`, `2024`) |>
  slice_sample(n = 10)
```

```
## # A tibble: 10 x 3
##    `2021`                                `2022`                          `2024`
##    <chr>                                 <chr>                           <chr>
##  1 keeping house                         other                           worki~
##  2 retired                               with a job, but not at work be~ keepi~
##  3 unemployed, laid off, looking for work <NA>                           <NA>
##  4 working full time                     unemployed, laid off, looking ~ worki~
##  5 working full time                     keeping house                   worki~
##  6 working full time                     working full time               in sc~
##  7 retired                               retired                         worki~
##  8 working full time                     <NA>                            <NA>
##  9 unemployed, laid off, looking for work working part time               <NA>
## 10 working full time                     <NA>                            <NA>
```

## 4.4 Recoding Values

Recoding helps combine categories or fix inconsistent entries.

```
# Combine 'separated' and 'divorced' into 'not married'
gssdat |>
  mutate(marital = recode(marital,
                          "separated" = "not married",
                          "divorced" = "not married")) |>
  count(marital)
```

```
##         marital     n
## 1       married 38959
## 2 never married 16963
## 3   not married 12706
## 4       widowed  7005
## 5          <NA>    66
```

```
# compare
gssdat |>
  count(marital)
```

```
##         marital     n
## 1      divorced 10158
## 2       married 38959
## 3 never married 16963
## 4     separated  2548
## 5       widowed  7005
## 6          <NA>    66
```

## 4.5 Career Readiness Connection

Data cleaning is a critical step in real-world research and professional work. The following career competencies were touched on in this section:

- **Technology**: Learning reproducible, code-based methods for preparing data.
- **Critical Thinking**: Deciding how to handle missing or inconsistent data involves judgment and awareness of the research context.
- **Equity and Inclusion**: Fair handling of missing responses and careful recoding can reduce bias in your analysis.

By mastering these skills, you will be able to prepare datasets for valid, trustworthy analysis. This is an essential ability in both academic research and data-driven careers.

## 4.6 Summary

In this chapter, we learned how to prepare data for analysis by:

- Inspecting the structure and contents of a dataset.
- Identifying and handling missing values.
- Reshaping data with `pivot_longer()` and `pivot_wider()`.
- Recoding categories when a research question requires different groupings.

Cleaning is not simply a technical step. Decisions about missing values, categories, and data structure can affect what questions we are able to answer and how we interpret the results.

## 4.7 Career Readiness Connection

The skills in this chapter support several career readiness competencies:

- **Technology**: Preparing data using reproducible, code-based workflows.
- **Critical Thinking**: Evaluating missing values, categories, and data-quality issues before analysis.
- **Professionalism**: Creating a documented and reproducible process for preparing data.
- **Equity and Inclusion**: Recognizing that decisions about missing responses and categories can affect whose experiences are represented in an analysis.

These skills are especially important in professional settings because analysts often spend substantial time preparing data before conducting statistical analyses.

### 4.7.1 Using AI to Study and Apply This Chapter

1. **Study the concepts:**
   `Explain the difference between missing data, incorrect data, and data that are simply not relevant to a particular analysis. Give me examples from survey research.`
2. **Practice R:**
   `Create a small, messy dataset containing missing values, inconsistent categories, and variables stored in the wrong shape. Give me a sequence of cleaning tasks to complete in R without giving me the solution immediately.`
3. **Debug R code:**
   `Here is my pivot_longer(), pivot_wider() or recode() code: [PASTE CODE]. Explain what each argument is doing and help me identify why my resulting dataset is not what I expected.`
4. **Evaluate an AI response:**
   `Give me an example of an AI recommendation for handling missing survey data that sounds reasonable but could introduce bias. Explain what questions I should ask before following the recommendation.`
5. **Connect to a career:**
   `How is data cleaning used in [MY FIELD]? Identify common data-quality problems professionals encounter and explain how reproducible R workflows could help address them. Provide sources I can verify.`

# 5 Visualizing and Summarizing Data

In this chapter, we learn to visualize and summarize data. You will learn about five types of graphs commonly used in the social sciences:

- Histograms
- Line graphs
- Scatterplots
- Bar charts
- Boxplots

These plots help us identify patterns, compare groups, and communicate findings clearly. Visualizations are especially useful for exploring relationships between variables, while tables are useful for summarizing large datasets. We will again use the General Social Survey (GSS) data throughout this chapter.

If you have not already done so, you can load the data and the `tidyverse` using the code below.

```
# load the package
library(tidyverse)

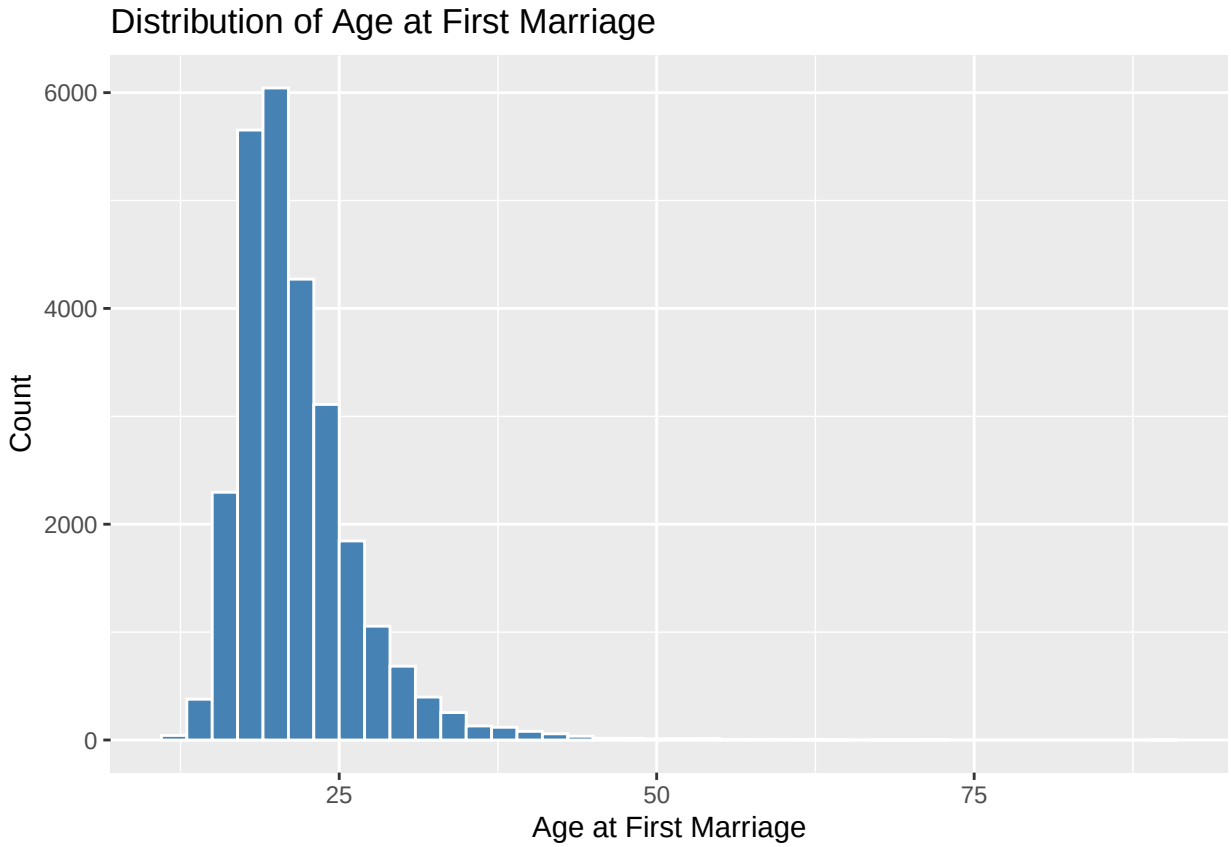
# load the data
gssdat <- read.csv("gssdat.csv")
```

## 5.1 Histograms: Distributions of Numerical Variables

A **histogram** displays how often each range of values occurs for a numerical variable. They are useful for numerical variables that take a wide range of values. They tell us about the center, shape, and spread of the distribution.

Here we examine the distribution of age at first marriage (`agewed`):

```
gssdat |>
  ggplot(aes(x = agewed)) +
  geom_histogram(binwidth = 2, fill = "steelblue", color = "white") +
  labs(title = "Distribution of Age at First Marriage",
       x = "Age at First Marriage",
       y = "Count")
```




We see that most respondents married in their early 20s. The histogram is slightly right skewed since the outliers lie to the right. In this case, the mean age at first marriage will be greater than the median age. We can impose the mean and median on the plots, but first we must compute them using `summarize`.

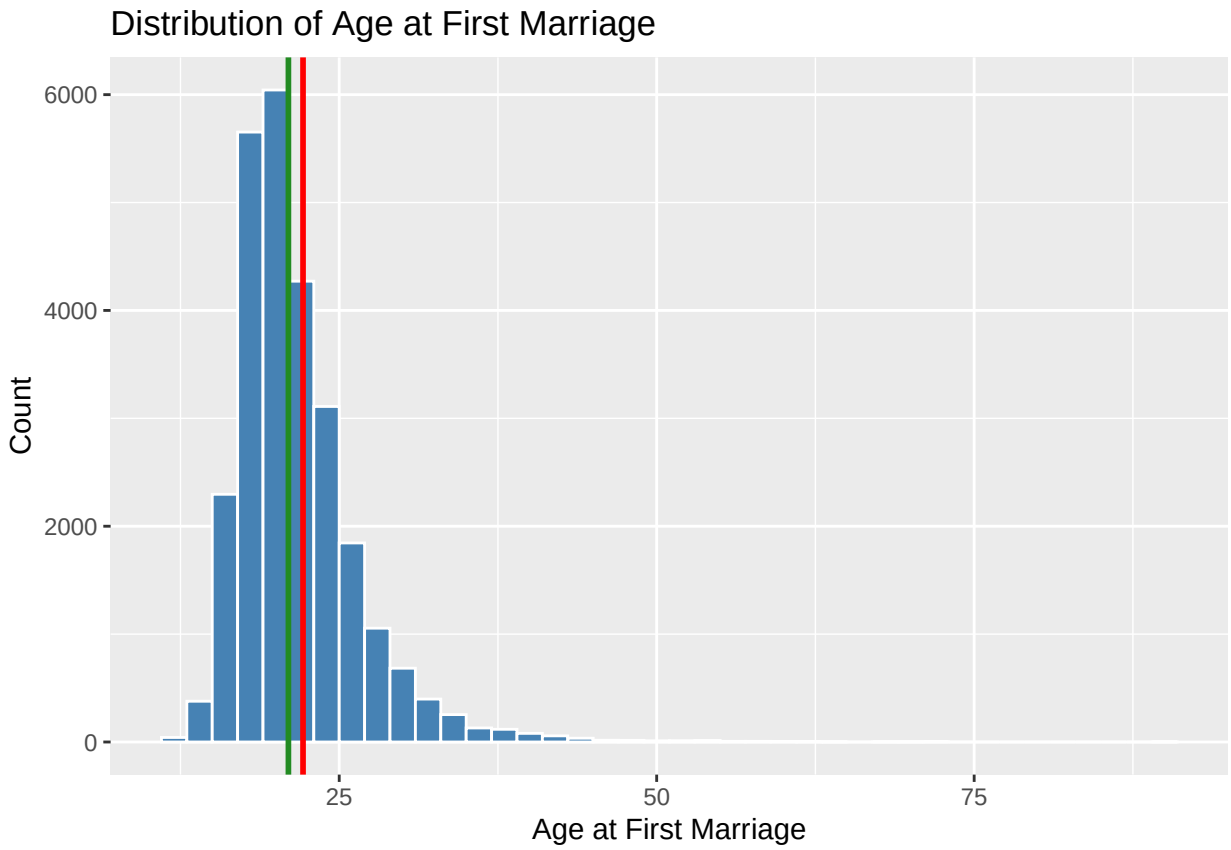
```
agewed_summary <- gssdat |>
  summarize(mean_age = mean(agewed, na.rm = TRUE),
            med_age = median(agewed, na.rm = TRUE))

agewed_summary
```

```
##   mean_age med_age
## 1 22.15006      21
```

To add the mean and median, we use `geom_vline`, which adds a vertical line to a plot at the specified x-intercept. For a horizontal line, we would use `geom_hline`. Since we are using two different datasets in the same plot, we need to specify the `agewed_summary` dataset in the `geom_vline` layer:

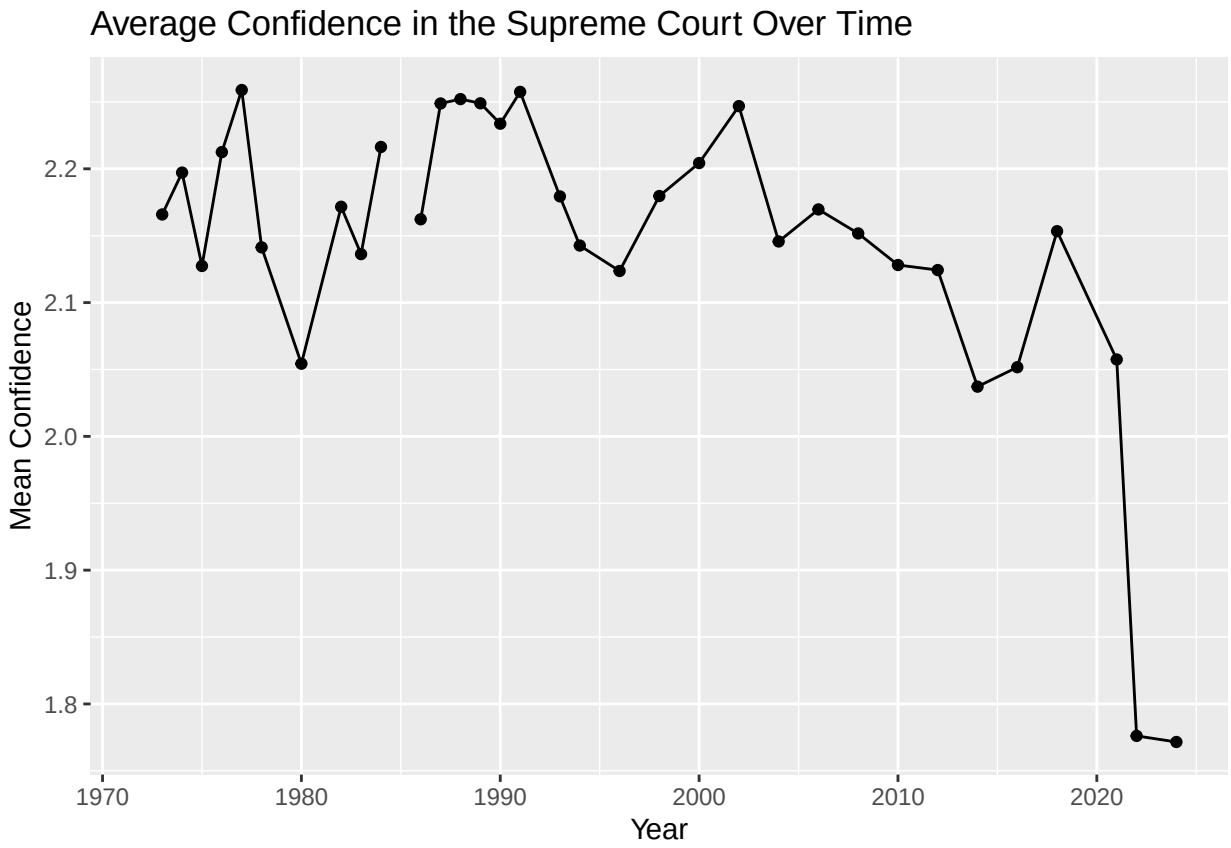
```
gssdat |>
  ggplot(aes(x = agewed)) +
  geom_histogram(binwidth = 2, fill = "steelblue", color = "white") +
  geom_vline(data = agewed_summary, aes(xintercept = mean_age), color = "red", linewidth = 1) +
  geom_vline(data = agewed_summary, aes(xintercept = med_age), color = "forestgreen", linewidth = 1) +
  labs(title = "Distribution of Age at First Marriage",
       x = "Age at First Marriage",
       y = "Count")
```




## 5.2 Line Graphs: Trends Over Time

**Line graphs** show how the value of a variable changes over time. For example, we can examine how average confidence in the U.S. Supreme Court (`conjudge`) has changed across survey years. We first mutate the data so that a one represents hardly any confidence and a three represents a great deal of confidence. Then, we calculate the summary table and pipe this into `ggplot`:

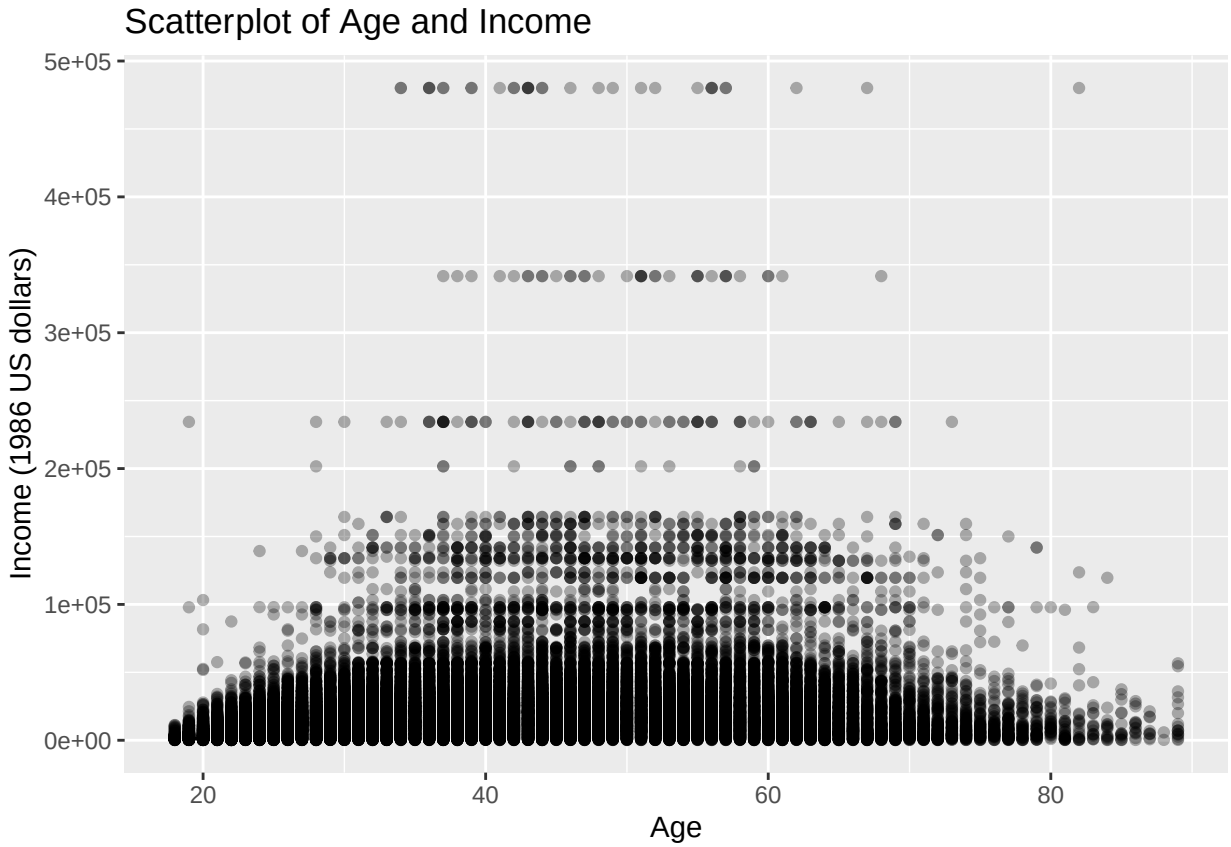
```
gssdat |>
  group_by(year) |>
  mutate(conjudge = recode(conjudge, `1` = 3L, `3` = 1L, `2` = 2L)) |>
  summarize(mean_conf = mean(conjudge, na.rm = TRUE)) |>
  ggplot(aes(x = year, y = mean_conf)) +
  geom_line() +
  geom_point() +
  labs(title = "Average Confidence in the Supreme Court Over Time",
       x = "Year",
       y = "Mean Confidence")
```




## 5.3 Scatterplots: Relationships Between Two Numerical Variables

**Scatterplots** help us examine relationships between two numerical variables. For example, we can compare age (`age`) and respondent income in constant 1986 US dollars (`realrinc`):

```
gssdat |>
  ggplot(aes(x = age, y = realrinc)) +
  geom_point(alpha = 0.3) +
  labs(title = "Scatterplot of Age and Income",
       x = "Age",
       y = "Income (1986 US dollars)")
```

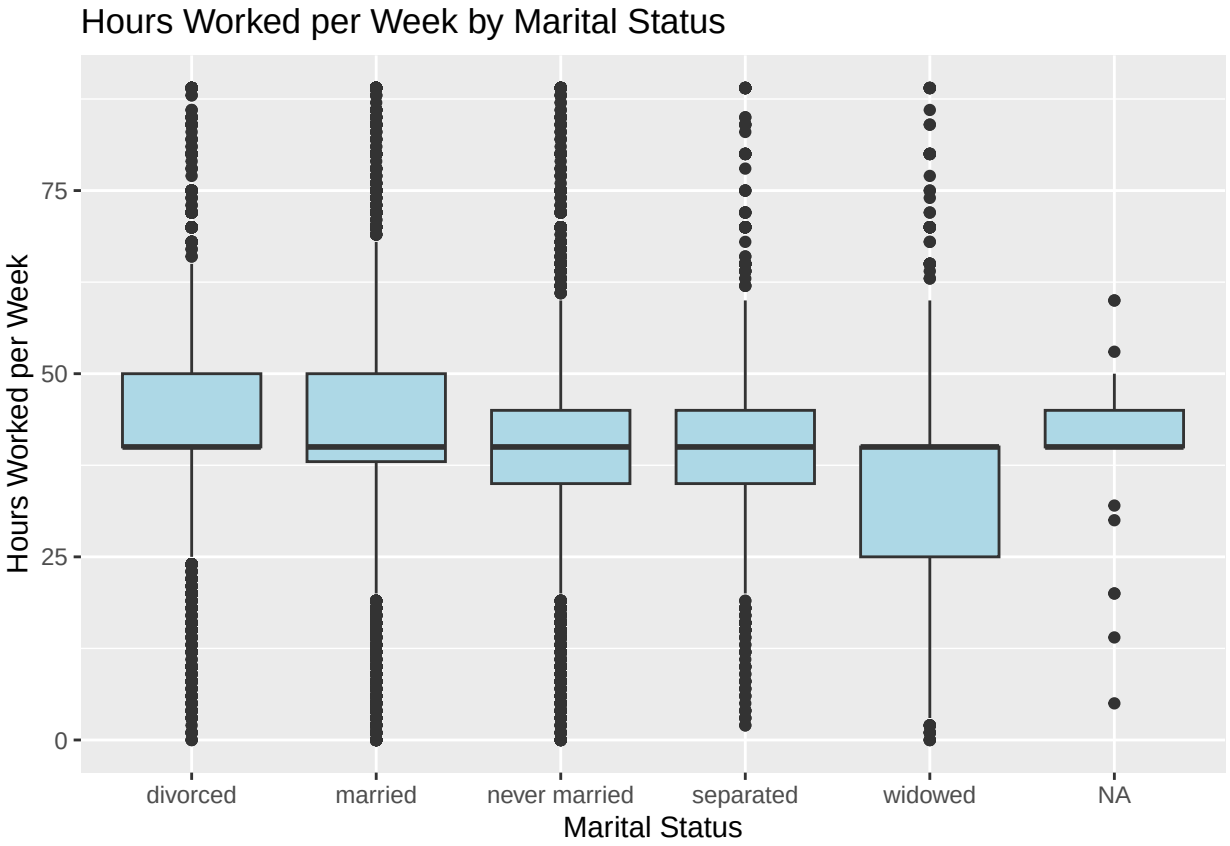



It seems that income is greater for respondents in their middle ages (30-70), while it is less for respondents whose ages are less than 30 or greater than 70.

## 5.4 Boxplots: Comparing Numerical Distributions Across Groups

**Boxplots** display the distribution of a numerical variable and are especially useful for comparing groups. We can compare medians (center bar), ranges (sides of box), and variability across categories. Outliers are plotted as actual points. Below we compare hours worked per week (`hrs1`) across marital status:

```
gssdat |>
  ggplot(aes(x = marital, y = hrs1)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Hours Worked per Week by Marital Status",
       x = "Marital Status",
       y = "Hours Worked per Week")
```




The distribution of hours worked is very similar for those never married and separated. It is a bit hard to determine which group has the highest work hours. Therefore, a plot like this would be well accompanied by the summary table below:

```
gssdat |>
  group_by(marital) |>
  summarize(avg_hrs = mean(hrs1, na.rm = TRUE)) |>
  arrange(avg_hrs)
```

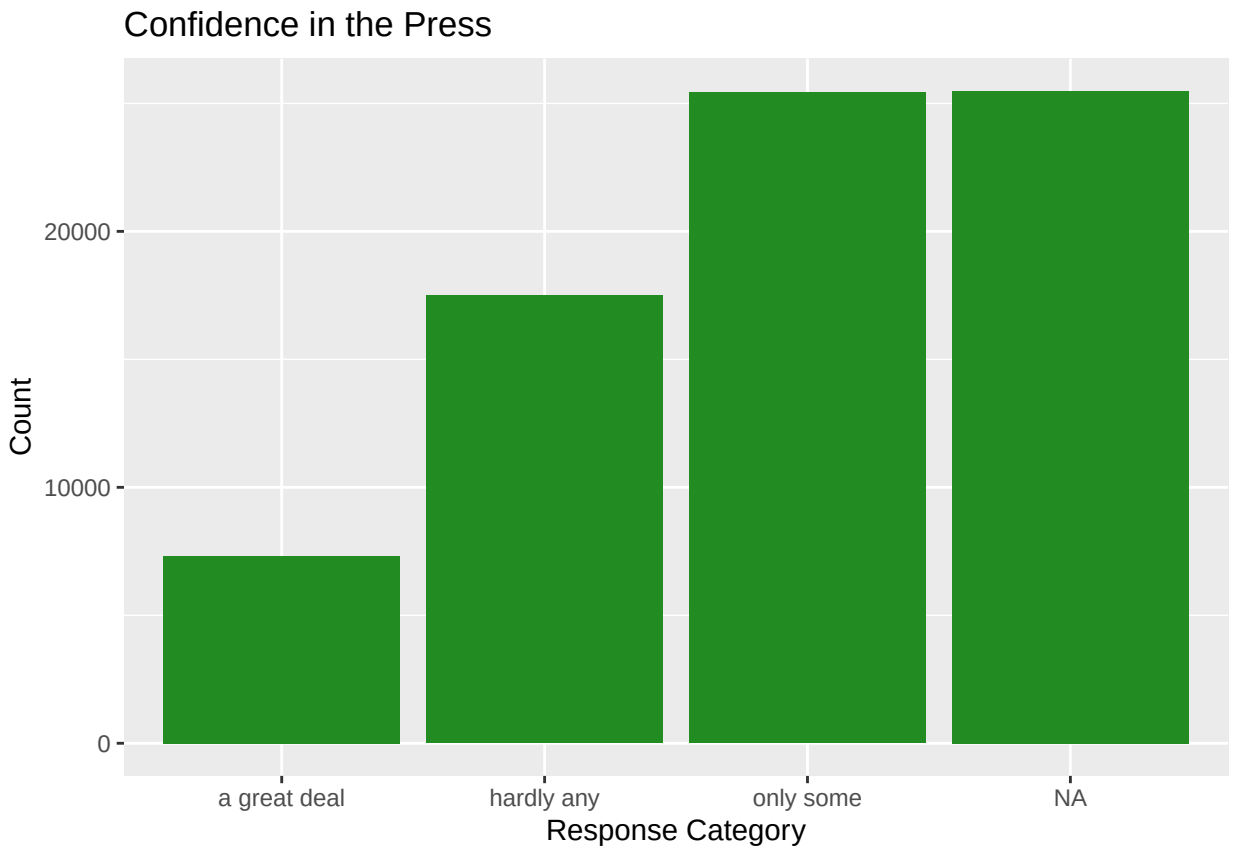
```
## # A tibble: 6 x 2
##   marital       avg_hrs
##   <chr>           <dbl>
## 1 widowed          35.3
## 2 <NA>             39.0
## 3 never married    40.0
## 4 separated        41.0
## 5 married          41.7
## 6 divorced         42.3
```

We see that divorced respondents reported the greatest working hours, on average, and those widowed reported the lowest.

## 5.5 Bar Charts: Summarizing Categorical Variables

**Bar charts** summarize counts or proportions for categorical variables. We look at confidence in the press (`conpress`) below:

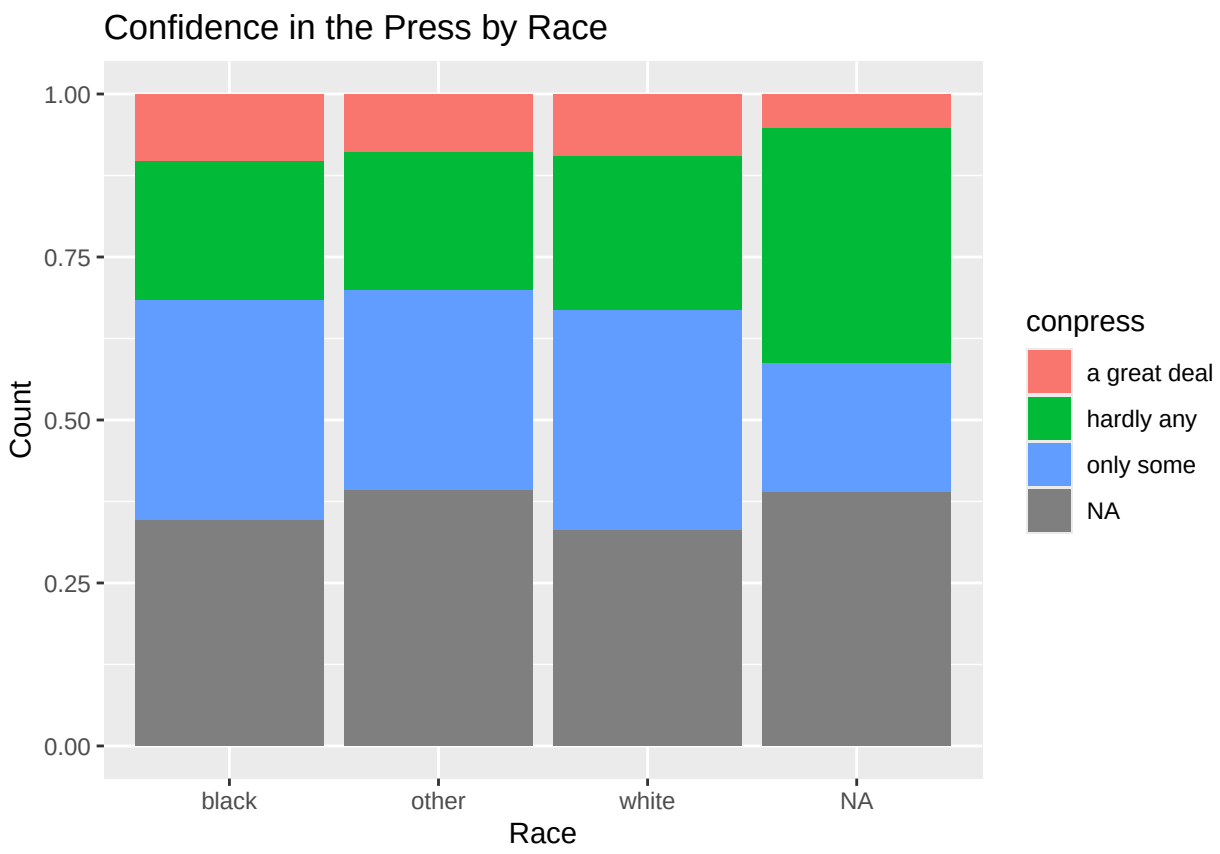
```
gssdat |>
  ggplot(aes(x = conpress)) +
  geom_bar(fill = "forestgreen") +
  labs(title = "Confidence in the Press",
       x = "Response Category",
       y = "Count")
```




## 5.6 Bar Charts with Group Comparisons

We can also compare responses across groups using side-by-side bar charts. For example, we examine confidence in the press by race (`race`) below:

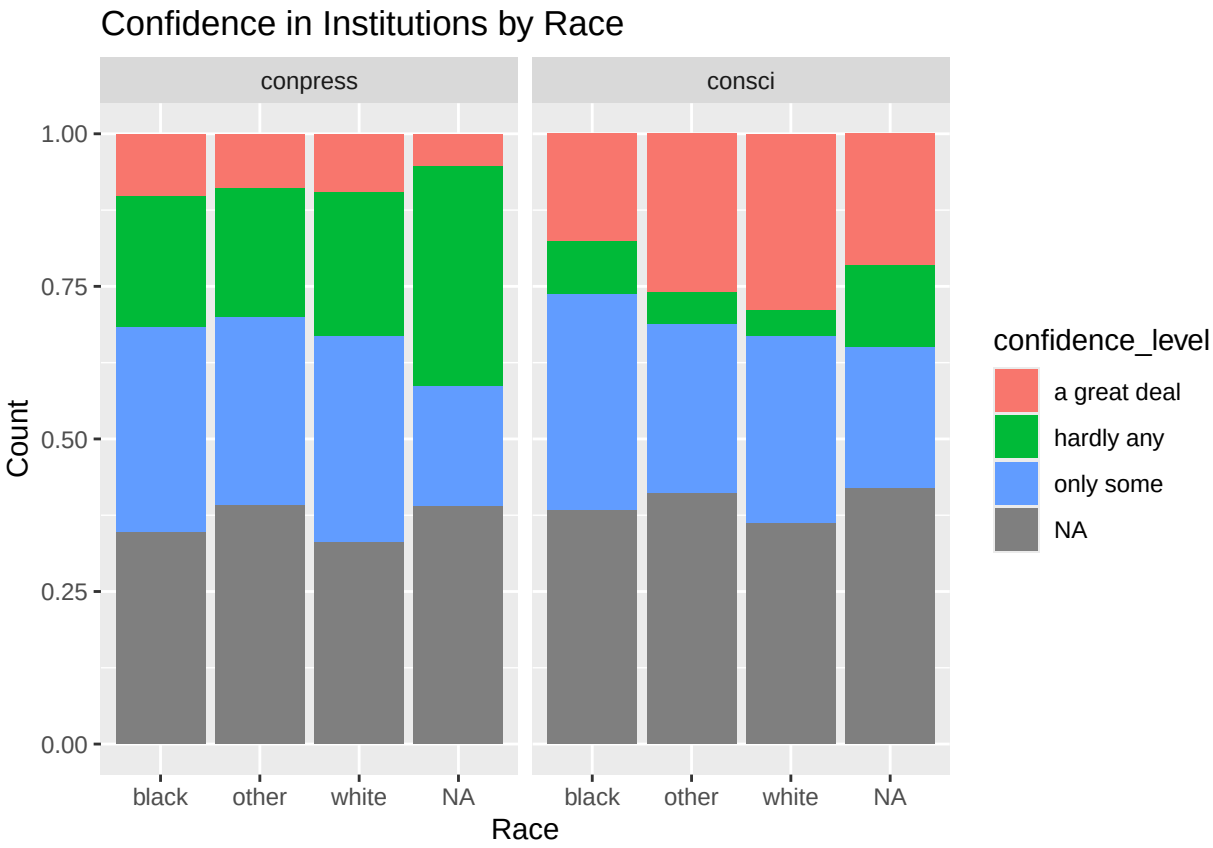
```
gssdat |>
  ggplot(aes(x = race, fill = conpress)) +
  geom_bar(position = "fill") +
  labs(title = "Confidence in the Press by Race",
       x = "Race",
       y = "Count")
```




## 5.7 Faceting: Small Multiples for Subgroup Comparisons

**Faceting** is a technique that creates multiple small plots for subgroups. Faceting is helpful when we want to examine subgroup differences without cluttering a single plot. For example, we reshape the dataset to analyze multiple confidence variables together. Below we use `pivot_longer` to gather several confidence variables into a long format before again making comparisons across races:

```
gssdat |>
  select(conpress, consci, race) |>
  pivot_longer(cols = c(conpress, consci),
               names_to = "confidence_type",
               values_to = "confidence_level") |>
  ggplot(aes(x = race, fill = confidence_level)) +
  geom_bar(position = "fill") +
  facet_wrap(~confidence_type) +
  labs(title = "Confidence in Institutions by Race",
       x = "Race",
       y = "Count")
```




There are higher rates of respondents indicating a great deal of confidence in science than in the press. The distribution also differs across racial groups, although missing responses should be considered when interpreting the plot. Faceting allows us to compare the distributions of two confidence variables across racial groups without placing all the information in a single plot.

## 5.8 Summary

In this chapter, we practiced creating and interpreting five core visualization types:

- Histograms for distributions of numerical variables.
- Line graphs for trends over time.
- Scatterplots for relationships between two numerical variables.
- Boxplots for comparing numerical distributions across categories.
- Bar charts for summarizing categorical data and comparing groups.

We also learned to:

- Gather related variables into long format for grouped visualizations.
- Use stacked bar charts to compare categories.
- Apply faceting to explore subgroup patterns.

These tools are essential for exploring data in the social sciences and for communicating findings clearly and responsibly.

## 5.9 Career Readiness Connection

The skills in this chapter support several career readiness competencies:

- **Communication**: Choosing graphs and summaries that communicate findings clearly.
- **Critical Thinking**: Looking for patterns, unusual observations, and differences before drawing conclusions.
- **Technology**: Using R to create reproducible visualizations and summaries.
- **Professionalism**: Designing figures that another person can understand without additional explanation.

Effective data visualization is not simply about making attractive graphs. It is about choosing a representation that helps an audience understand evidence relevant to a particular question.

### 5.9.1 Using AI to Study and Apply This Chapter

1. **Study the concepts:**
   `Quiz me on when to use a histogram, line graph, scatterplot, bar chart, and boxplot. Give me one research question at a time and ask me to choose the appropriate visualization before explaining the answer.`
2. **Practice R:**
   `Give me three GSS-style research questions and ask me to choose an appropriate ggplot2 visualization for each. Do not give me the code until I explain why I chose each graph.`
3. **Debug R:**
   `Here is my ggplot2 code: [PASTE CODE]. Explain what each aesthetic and layer is doing, identify any errors, and suggest how I can improve the graph without changing the research question.`
4. **Critically evaluate a graph:**
   `Describe a misleading statistical visualization involving a bar chart, histogram, or boxplot. Ask me to identify what makes it misleading before explaining the problem.`
5. **Connect to a career:**
   `Find three examples of how data visualization is used in [MY FIELD]. Explain what audiences those visualizations are designed for and what makes the communication effective. Give me sources to verify.`

# 6 Methods for the Mean

In the previous chapter, we used graphs and summary statistics to explore data and identify interesting patterns. For example, we compared groups, examined relationships between variables, and summarized important characteristics of the General Social Survey data. While these descriptive methods are valuable, they naturally lead to a deeper question: *Do the patterns we observe in our sample likely reflect the population, or could they simply be the result of random sampling?*

Imagine surveying 100 people about their weekly work hours. If you repeated the survey with a different group of 100 people, you would almost certainly obtain a different average. This natural variation from one sample to another is known as *sampling variability*. Statistical methods use probability to account for this variability, allowing us to evaluate how strongly the observed data support a hypothesis about a population.

In the next few chapters, you will learn how to use statistical methods to answer questions about means, proportions, medians, and relationships between variables. Each analysis follows the same general workflow: begin with a research question, explore the data, identify an appropriate statistical method, perform the analysis, and interpret the result in context.

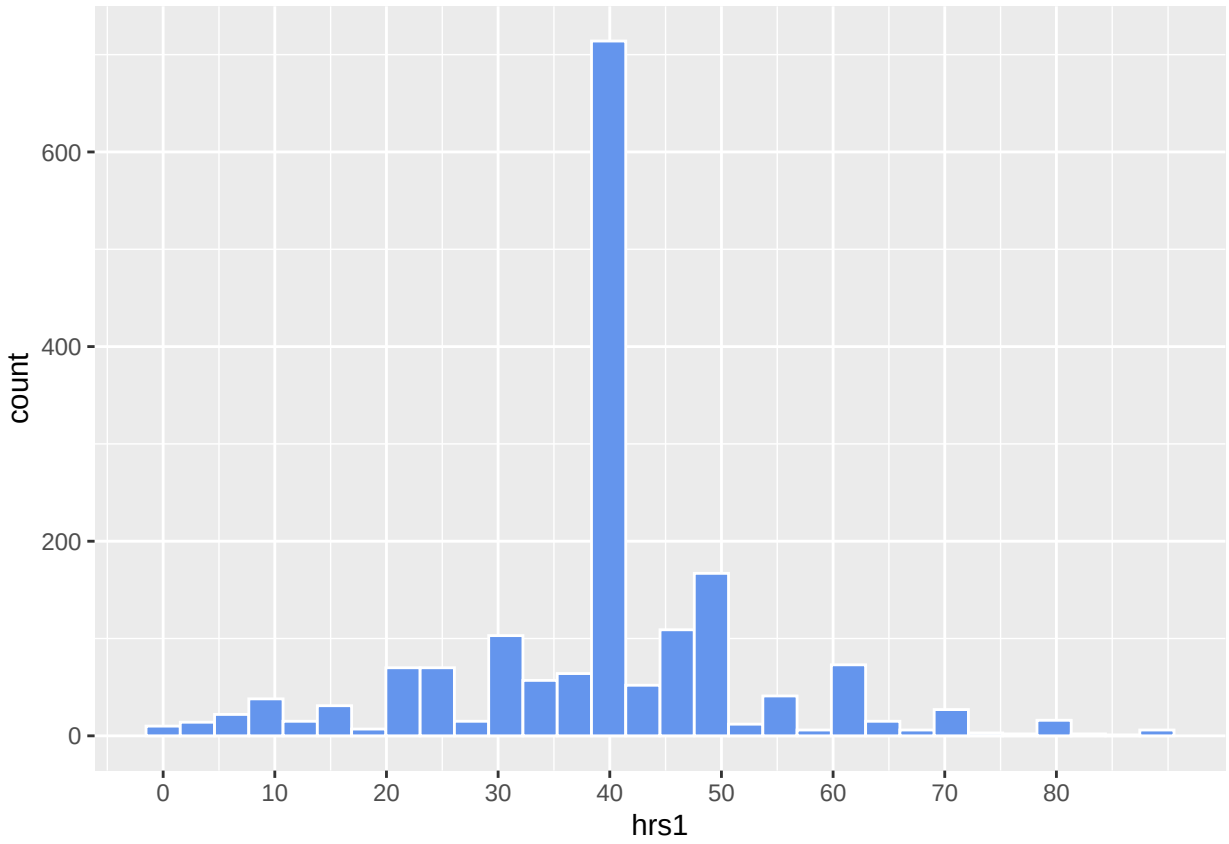
Before we begin, we subset the dataset to survey year 2024 to use more recent data.

```
gssdat_sub <- gssdat |>
  filter(year == 2024)
```

## 6.1 One-Sample $t$-Test for Mean

The example provided earlier - which concerned surveying 100 people about their weekly work hours - is often encountered in real life by researchers studying labor patterns. To understand work hours, we could compare average weekly work hours with a commonly used benchmark. For example, we might ask, *"Is the average number of hours worked different from the standard 40-hour work week?"* To answer this question we will use the variable `hrs1`, which represents the number of hours worked last week by respondents. We can visualize the distribution of `hrs1` using the code below.

```
gssdat_sub |>
  ggplot(aes(x = hrs1)) +
  geom_histogram(fill = "cornflowerblue", color = "white") +
  scale_x_continuous(breaks = seq(0, 80, 10))
```




Based on the sample alone, does it seem that the average number of hours worked by respondents differs from 40? Examining the very apparent center of the distribution, it seems that the data do not provide evidence for a different average number of hours worked than 40. To account for sampling variability though, we can further analyze the data using a one-sample t-test.

The `t.test` function is a multi-purpose function for running different types of t-tests in RStudio. Using the code below, we can access the help page for the function and determine its syntax:

```
?t.test
```

```
t.test(x, y = NULL,
       alternative = c("two.sided", "less", "greater"),
       mu = 0, paired = FALSE, var.equal = FALSE,
       conf.level = 0.95, ...)
```

The general syntax for the function includes many arguments. For now, we will use `x`, `alternative`, `mu`, and `conf.level`. These specify the data, direction of the alternative hypothesis, hypothesized mean, and confidence level, respectively.

Since we want to determine whether the average number of hours worked is *different* from 40, we set `alternative = "two.sided"` and `mu = 40`. The argument `conf.level = 0.95` will produce a 95% confidence interval, so that the significance level is 0.05.

```
t.test(gssdat_sub$hrs1, alternative = "two.sided", mu = 40, conf.level = 0.95)
```

```
##
##  One Sample t-test
```

```
##
## data:  gssdat_sub$hrs1
## t = -1.6969, df = 1767, p-value = 0.08988
## alternative hypothesis: true mean is not equal to 40
## 95 percent confidence interval:
##  38.79286 40.08723
## sample estimates:
## mean of x
##  39.44005
```

The output of `t.test` includes the test statistic, p-value, and confidence interval. The test statistic is `t = -1.6969`. The p-value is greater than the significance level, and the confidence interval includes 40. Given these results, we conclude that there is *no evidence* that the mean number of hours worked is different from 40. With a p-value greater than 0.05, we fail to reject the null hypothesis that the mean is 40 hours. We are 95% confident that the mean number of hours worked is between 38.79 and 40.09 hours.

### 6.1.1 Practical vs Statistical Significance

*Statistical significance* - which is determined by a p-value - tells us whether an observed result is unlikely to have occurred by chance, while *effect size* describes the magnitude of a difference or relationship observed in the data. A result can be statistically significant but have a very small effect size, meaning that the difference may not be practically important or meaningful in real life contexts.

Given that we have a large sample size, it is possible to obtain a statistically significant result simply due to the amount of data available. We did not obtain a significant result in this case, but we can still calculate the effect size. Subtracting 40 from the sample mean number of hours worked, we see that the difference is only about 0.56 hours, which is approximately 34 minutes after rounding. This confirms that the results are both statistically and practically insignificant.

```
abs(mean(gssdat_sub$hrs1, na.rm = TRUE) - 40)
```

```
## [1] 0.5599548
```

## 6.2 Paired $t$-Test for Mean Difference

Sometimes the question we want to answer involves two measurements taken from the same family, person, or other matched unit. In that situation, treating the two measurements as if they were unrelated would ignore the connection between them. A paired $t$-test is useful when each observation in one variable can be naturally matched with exactly one observation in the other variable.

Consider educational attainment across generations. We can ask: *Do respondents generally have higher levels of educational attainment than their fathers?* The GSS includes both respondents' educational attainment (`educ`) and their fathers' educational attainment (`paeduc`), measured in years of education. Because the two measurements belong to the same respondent, the observations are paired.

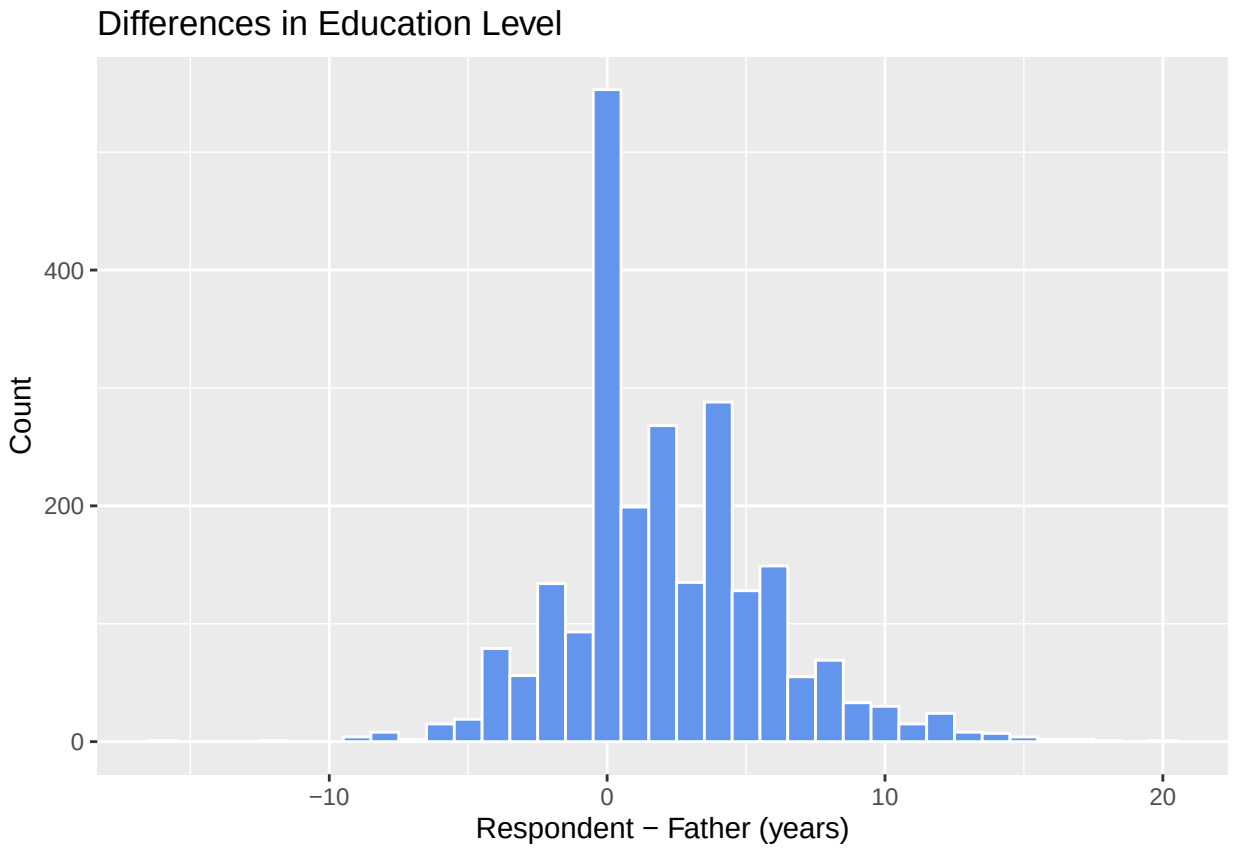
Before conducting a statistical test, we should first explore the data. A useful way to begin is to look at the two distributions and then examine the difference between the two measurements for each respondent. To prepare the data we remove observations with missing values for `educ` and `paeduc` and then add a variable for the difference.

```
educ_pair <- gssdat_sub |>
  select(educ, paeduc) |>
  filter(!is.na(educ), !is.na(paeduc)) |>
  mutate(educ_diff = educ - paeduc)
```

The `educ_diff` variable represents the difference between a respondent's education and their father's education. A positive value means the respondent has more years of education than their father, while a negative value means the respondent has fewer.

A histogram of these differences gives us a useful first look at the question. If the distribution is centered above zero, respondents generally have more education than their fathers. If it is centered near zero, the two generations have similar levels of educational attainment. The distribution is shifted above 0 indicating that most respondents have a greater education level than their father.

```
educ_pair |>
  ggplot(aes(x = educ_diff)) +
  geom_histogram(binwidth = 1, fill = "cornflowerblue", color = "white") +
  labs(title = "Differences in Education Level",
       x = "Respondent - Father (years)",
       y = "Count")
```




We can also summarize the two variables and the difference directly.

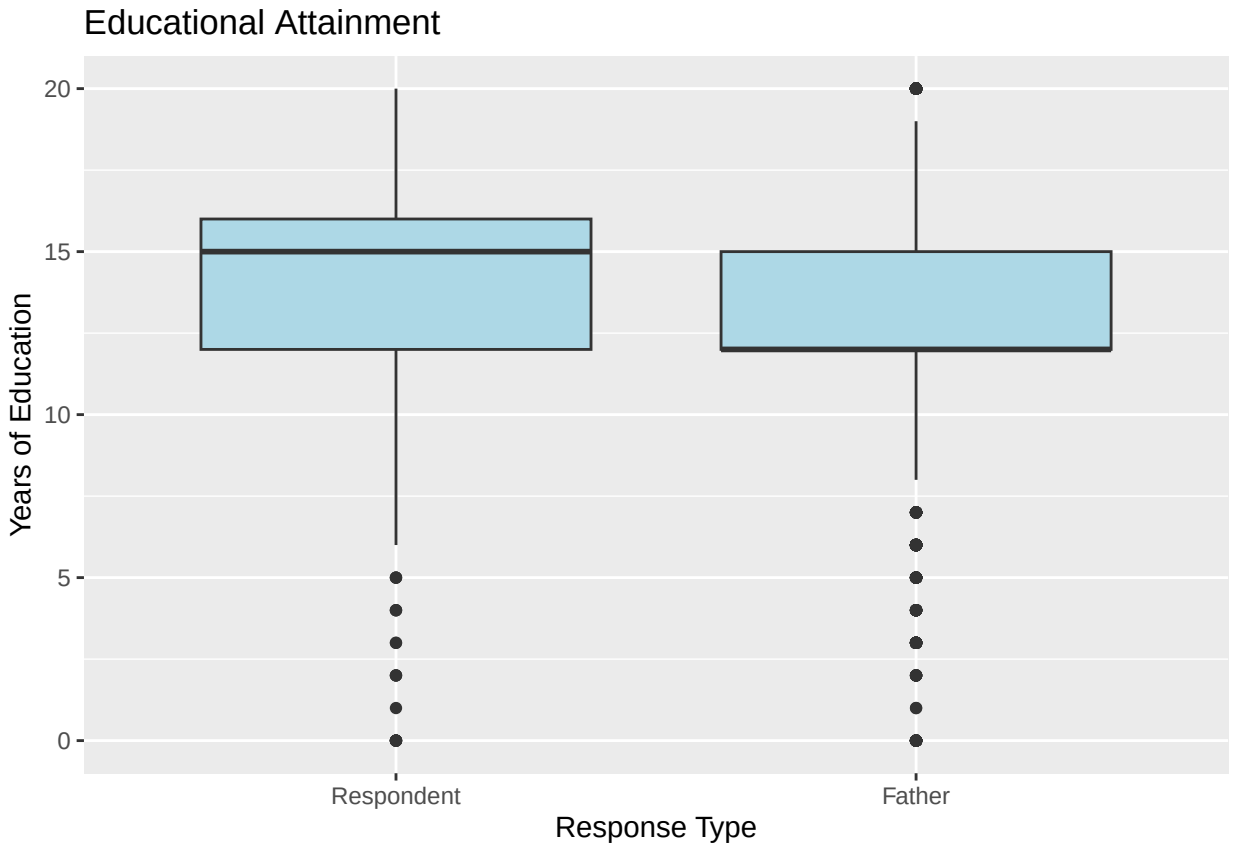
```
educ_pair |>
  summarize(
    mean_educ = mean(educ),
    median_educ = median(educ),
    mean_paeduc = mean(paeduc),
    median_paeduc = median(paeduc),
    mean_difference = mean(educ_diff),
    median_difference = median(educ_diff),
    sd_difference = sd(educ_diff)
  )
```

```
##   mean_educ median_educ mean_paeduc median_paeduc mean_difference
## 1  14.59085          15    12.48972            12        2.101133
##   median_difference sd_difference
## 1                 2      3.743263
```

The key summary for a paired analysis is the difference. The mean difference tells us how many more or fewer years of education respondents have, on average, than their fathers. The median difference provides another description of the typical change and can be useful if the distribution of differences is skewed. On average, respondents completed about 2 more years of education than their fathers. Notice that the difference in variable means is equal to the mean of the difference variable.

We can also visualize the two distributions directly. A boxplot places the distributions side by side, while the difference histogram focuses on the quantity that the paired test analyzes. Comparing the amount of overlap between these boxplots is a simple way to determine how similar the distributions are. In this case there is much more overlap than non-overlap indicating moderate differences.

```
educ_pair |>
  select(educ, paeduc) |>
  pivot_longer(cols = c(educ, paeduc),
               names_to = "response_type",
               values_to = "years") |>
  ggplot(aes(x = response_type, y = years)) +
  geom_boxplot(fill = "lightblue") +
  scale_x_discrete(labels = c("Respondent", "Father")) +
  labs(title = "Educational Attainment",
       x = "Response Type",
       y = "Years of Education")
```



The paired $t$-test asks whether the population mean difference is different from zero. In other words, instead of testing two unrelated means, we test the mean of the `educ_diff` variable. We can conduct the test directly by supplying the two paired variables to `t.test()` and setting `paired = TRUE`. The null hypothesis is that the mean difference in educational attainment is zero, so we set `mu = 0`. The alternative hypothesis is that the mean difference is not zero, so we set `alternative = "two.sided"`.

```
t.test(educ_pair$educ,
       educ_pair$paeduc,
       paired = TRUE,
       alternative = "two.sided",
       mu = 0,
       conf.level = 0.95)
```

```
##
##  Paired t-test
##
## data:  educ_pair$educ and educ_pair$paeduc
## t = 27.401, df = 2382, p-value < 2.2e-16
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  1.950764 2.251502
## sample estimates:
## mean difference
##        2.101133
```

The output provides a test statistic, degrees of freedom, p-value, confidence interval, and an estimate of the mean difference. The p-value should be interpreted in relation to the null hypothesis rather than as the probability that the null hypothesis is true. In this case there is very strong evidence that the mean difference is not equal to zero, and we reject the null hypothesis at the 0.05 significance level.

A 95% confidence interval for the mean difference gives us a range of plausible values for the population mean difference. Since the interval does not include zero, this provides evidence against a population mean difference of zero at the corresponding 5% significance level. With 95% confidence, the mean difference in years of education is between 1.95 and 2.25 years.

The paired structure is important here. We are not asking whether the average differs between two independent groups. We are asking whether the average difference between two dependent variables is different from zero. Next, we will consider a different situation in which the two groups are independent.

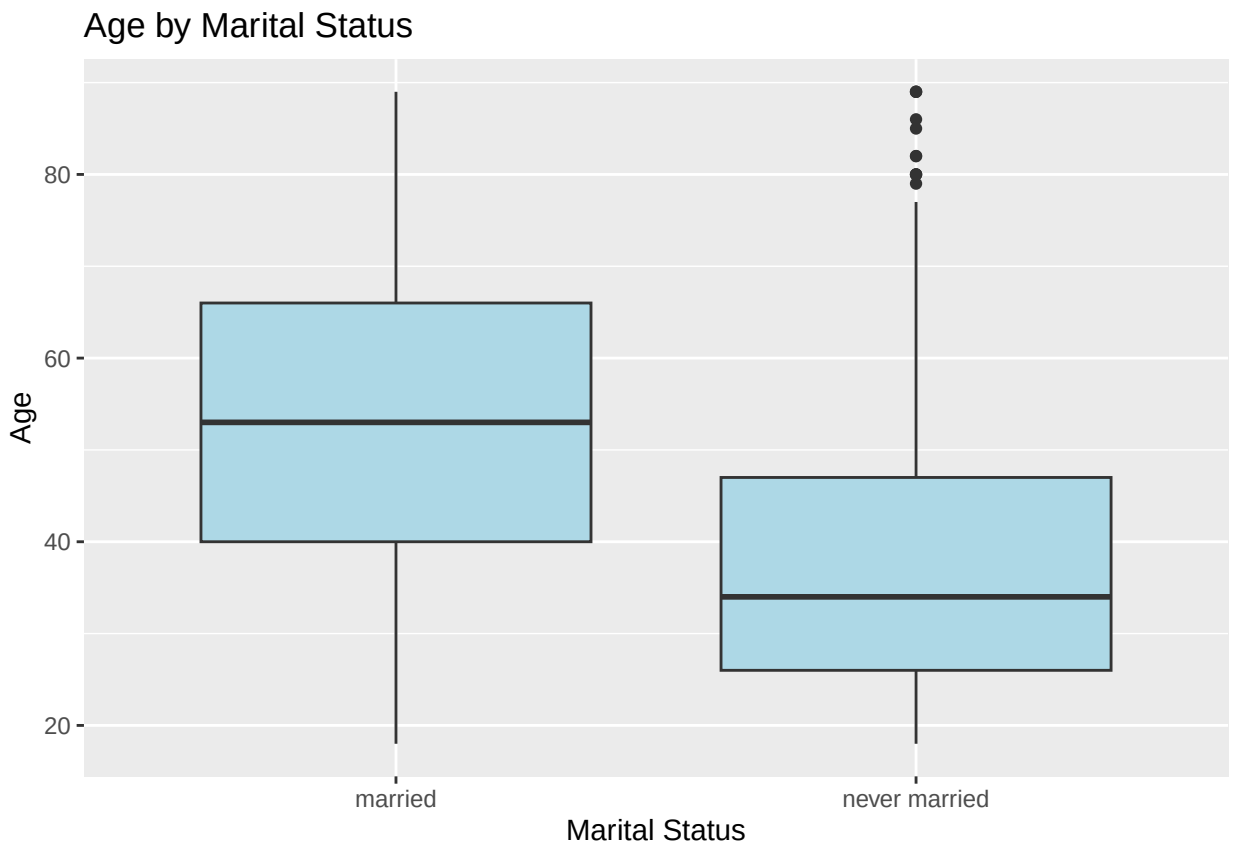
## 6.3 Two-Sample $t$-Test for Difference in Means

Suppose we want to investigate whether age differs between respondents who are currently married and respondents who have never married. The GSS variable `marital` identifies marital status, while `age` records the respondent's age. The observations in the married group are not naturally paired with observations in the never-married group, so a paired test would not be appropriate.

Our question is: *Do respondents who are currently married differ in average age from respondents who have never married?* Before testing this question, we should inspect the distributions in the two groups. A boxplot makes it easy to compare the center and spread and identify unusual observations.

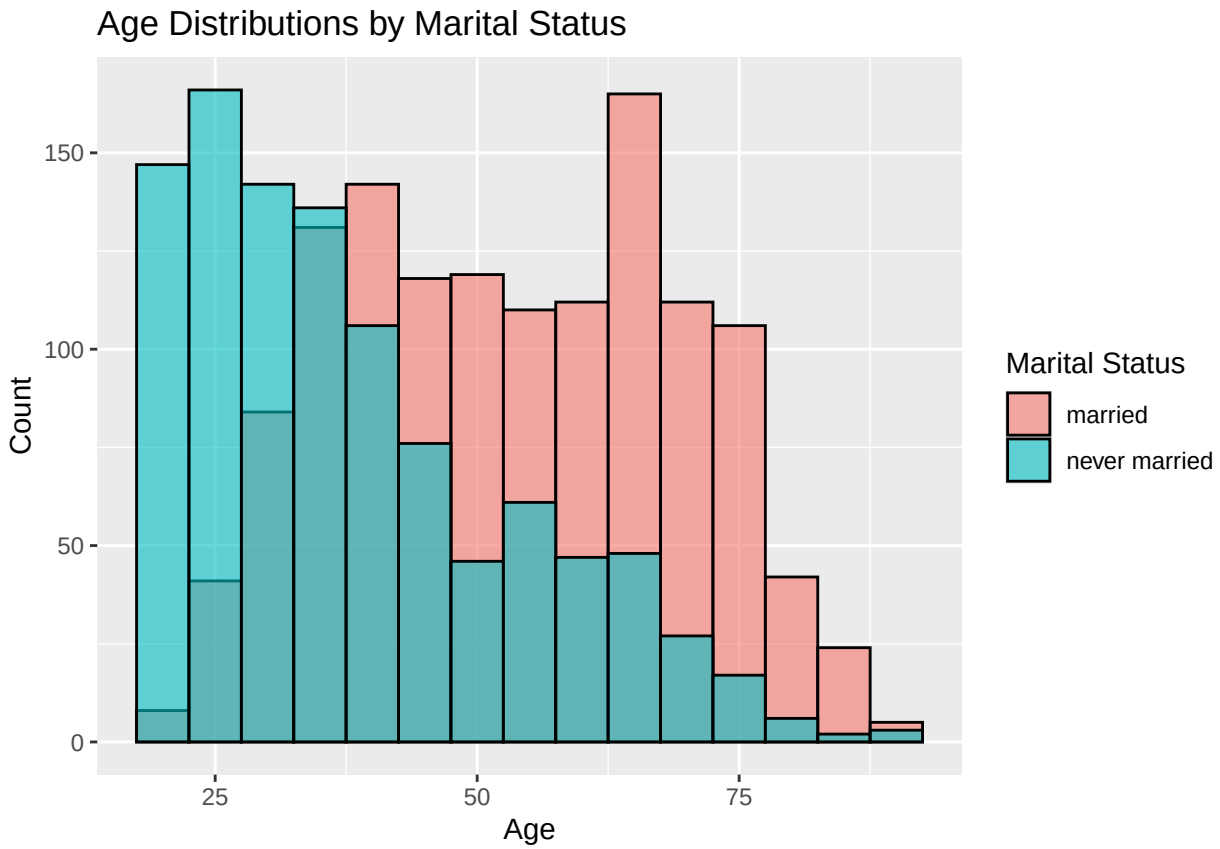
```
age_marital <- gssdat_sub |>
  filter(marital %in% c("married", "never married"),
         !is.na(age))
```

```r
age_marital |>
  ggplot(aes(x = marital, y = age)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Age by Marital Status",
       x = "Marital Status",
       y = "Age")
```




We can also examine the distributions using overlaid histograms.

```r
age_marital |>
  ggplot(aes(x = age, fill = marital)) +
  geom_histogram(binwidth = 5, alpha = 0.6, position = "identity", color = "black") +
  labs(title = "Age Distributions by Marital Status",
       x = "Age",
       y = "Count",
       fill = "Marital Status")
```




These plots help us identify whether the groups have noticeably different centers and whether one group has substantially greater variability than the other. The variability is about the same for married and never married respondents. However, the center of the distribution for never married respondents is clearly shifted towards lesser values in both figures.

To confirm what we are seeing we can calculate descriptive statistics separately for each group.

```
age_marital |>
  group_by(marital) |>
  summarize(
    n = n(),
    mean_age = mean(age),
    median_age = median(age),
    sd_age = sd(age)
  )
```

```
## # A tibble: 2 x 5
##   marital           n mean_age median_age sd_age
##   <chr>         <int>    <dbl>      <dbl>  <dbl>
## 1 married        1319     53.4         53   15.9
## 2 never married  1030     38.3         34   15.5
```

The difference in sample means for the married and never-married respondents is substantially different from zero. The inferential question is whether a difference of this size would be unusual if the population means were equal.

For two independent groups, we use a two-sample $t$-test. The default `t.test()` procedure in R uses the unequal-variance, or Welch, version of the test. This approach does not require us to assume that the two

populations have the same variance and is therefore a useful general method for comparing two independent means.

The formula `age ~ marital` tells R that age is the numerical outcome and marital status defines the two groups. We will use the formula syntax `response ~ explanatory` in other functions as well. The null hypothesis is that the two population means are equal. The alternative hypothesis is that the population means differ.

```
t.test(age ~ marital,
       data = age_marital,
       alternative = "two.sided",
       var.equal = FALSE,
       conf.level = 0.95)
```

```
##
##  Welch Two Sample t-test
##
## data:  age by marital
## t = 23.082, df = 2232.2, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group married and group never married is no
## 95 percent confidence interval:
##  13.76658 16.32302
## sample estimates:
##       mean in group married mean in group never married
##                    53.35936                    38.31456
```

The mean age among married respondents was 53.36 years, compared with 38.32 years among respondents who had never married. The p-value from the equal variance two-sample $t$-test provides strong evidence of a difference in the population mean age between married and unmarried respondents. The 95% confidence interval indicates that the mean age for married individuals was 13.76 to 16.33 years older than that for never married individuals.

The confidence interval describes plausible values for the difference between the population means. We must pay attention to the direction of the difference when interpreting the interval. Since R uses the ordering of the grouping variable, we should always check which group is subtracted from which before writing the conclusion.

## 6.4 ANOVA and Pairwise $t$-tests

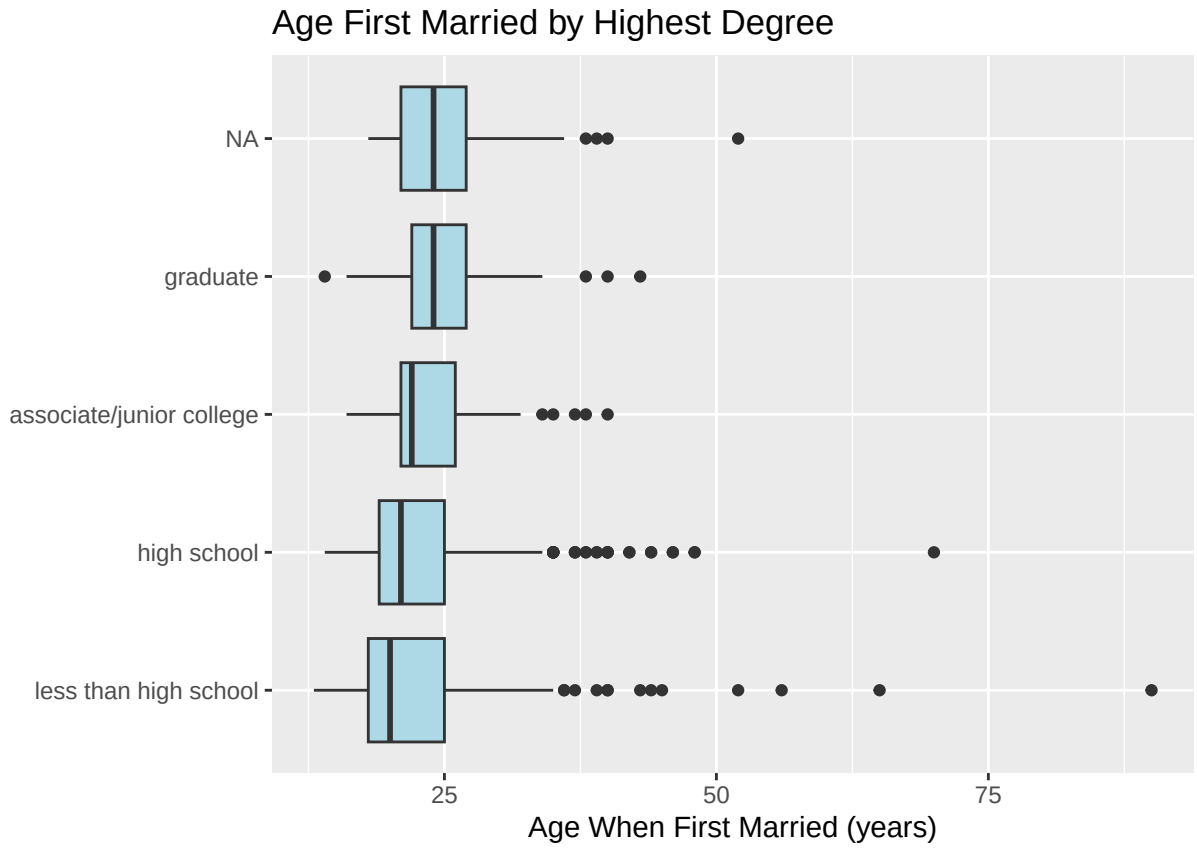
A two-sample $t$-test is designed for comparing two groups. What happens when the explanatory variable has more than two groups?

Suppose we want to investigate whether age when first married differs based on highest degree earned. The GSS variable `agewed` indicates respondents' age when first married, while `degree` identifies a respondent's highest degree earned. The `agewed` survey item was dropped in the early 90's, except for its reintroduction for the 2006 survey year. Therefore, we will use the 2006 data to analyze how `agewed` differs across degree levels.

Rather than conducting a separate $t$-test for every possible degree level, we can begin with a one-way analysis of variance, or ANOVA. Our question is: *Is there evidence that average age when first married differs across degree levels?* Before fitting the model, examine the distribution of age when first married across degree levels. We use factor to order the levels of `degree` since it is an ordinal categorical variable.

```r
age_wed_sub <- gssdat |>
  filter(year == 2006, !is.na(agewed), !is.na(degree)) |>
  mutate(degree = factor(degree, levels = c("less than high school", "high school",
                                            "associate/junior college", "bachelors",
                                            "graduate")))

age_wed_sub |>
  ggplot(aes(x = degree, y = agewed)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Age First Married by Highest Degree",
       x = "",
       y = "Age When First Married (years)") +
  coord_flip()
```




We can also calculate descriptive statistics to better understand what group differences exist before performing an ANOVA.

```r
age_wed_sub |>
  group_by(degree) |>
  summarize(n = n(),
            mean_age = mean(agewed),
            median_age = median(agewed),
            sd_age = sd(agewed))
```

```
## # A tibble: 5 x 5
##   degree                      n mean_age median_age sd_age
```

```
##   <fct>                    <int>    <dbl>      <dbl>  <dbl>
## 1 less than high school      195     22.5         20   8.95
## 2 high school                590     22.9         21   5.64
## 3 associate/junior college    86     23.7         22   4.86
## 4 graduate                   104     25.1         24   4.82
## 5 <NA>                       185     24.6         24   4.61
```

As degree level increases from less than high school to graduate, there is a clear increase in both the median and average age when first married. An ANOVA tests whether there is evidence that at least one of the population means differs. The null hypothesis is that all population means are equal. The alternative hypothesis is that at least one population mean differs from another. We can fit the ANOVA model using `aov()`.

```
wed_anova <- aov(agewed ~ degree, data = age_wed_sub)

summary(wed_anova)
```

```
##              Df Sum Sq Mean Sq F value  Pr(>F)
## degree        3    556  185.45   4.657 0.00307 **
## Residuals   971  38667   39.82
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 185 observations deleted due to missingness
```

The ANOVA produces an $F$ statistic and p-value. The $F$ statistic compares variation between groups to variation within groups. If the group means are very different relative to the amount of variation within groups, the $F$ statistic will be much larger than 1. Otherwise, it will be close to or less than 1 indicating similar variability within and between groups.

In this case there is very strong evidence that the average age when first married differs for at least one degree level. However, a statistically significant ANOVA does **not** tell us which groups differ. It tells us that the data are inconsistent with a model in which every population mean is the same. We therefore need to determine where the differences occur.

### 6.4.1 Pairwise Comparisons

When the overall ANOVA provides evidence that group means are not all equal, we can conduct pairwise comparisons. However, repeatedly conducting ordinary two-sample $t$-tests can inflate the probability of incorrectly rejecting the null hypothesis. We can use a Bonferroni correction to adjust the overall significance level by dividing the significance level, or multiplying the $p$-value, by the number of possible comparisons. If there are $k$ groups then the number of possible comparisons is $k(k-1)/2$.

We use the `pairwise.t.test` function to test the null hypotheses that each pair of group means is the same versus a two-sided alternative. Setting the argument `p.adjust.method = bonferroni` performs the correction and returns an adjusted $p$-value.

```
pairwise.t.test(x = age_wed_sub$agewed, g = age_wed_sub$degree,
                alternative = "two.sided", p.adjust.method = "bonferroni")
```

```
##
##  Pairwise comparisons using t tests with pooled SD
##
```

```
## data:  age_wed_sub$agewed and age_wed_sub$degree
##
##                           less than high school high school
## high school               1.0000                -
## associate/junior college 0.7712                1.0000
## graduate                  0.0038                0.0056
##                           associate/junior college
## high school               -
## associate/junior college -
## graduate                  0.7915
##
## P value adjustment method: bonferroni
```

There is significant evidence that the average age when first married differs between individuals with a bachelor's or graduate degree and individuals with less than a high school or a high school education. We can visualize the group means to help connect the statistical output to the original question.

ANOVA is useful because it gives us a single overall test before we investigate individual group differences. It helps prevent us from treating every possible pairwise comparison as a separate primary test. However, a statistically significant ANOVA does not tell us why groups differ. It also does not establish that changes in degree level cause changes in age when first married. The data are observational, and group membership may be related to many other characteristics.

The interpretation should therefore distinguish three questions: Do the sample means differ? Is there statistical evidence that population means are not all equal? Which groups appear to differ, and how large are those differences? These are related but different questions.

## 6.5 Summary

In this chapter, we used statistical methods for questions about means:

- A one-sample $t$-test compares a population mean with a hypothesized value.
- A paired $t$-test evaluates the mean difference between matched observations.
- A two-sample $t$-test compares means from two independent groups.
- ANOVA compares means across more than two groups.
- Pairwise comparisons help identify which groups differ after an overall ANOVA provides evidence of differences.

For every method, the goal is the same: connect the statistical procedure to a specific research question and interpret the result in context.

## 6.6 Career Readiness Connection

The skills in this chapter support several career readiness competencies:

- **Critical Thinking**: Selecting a statistical method based on the research design and variables.
- **Technology**: Implementing statistical analyses reproducibly in R.
- **Communication**: Translating statistical output into conclusions that a non-statistical audience can understand.
- **Professionalism**: Reporting uncertainty rather than presenting a sample difference as a definitive population claim.

### 6.6.1 Using AI to Study and Apply This Chapter

1. **Study method selection:**
   `Give me five research questions involving means. Ask me to decide whether each requires a one-sample t-test, paired t-test, two-sample t-test, or ANOVA. Do not give me the answers until I explain my reasoning.`
2. **Practice R:**
   `Create a small dataset and give me a research question that requires a t-test. Ask me to identify the variables, visualize the data, choose the test, and write the R code. Give feedback only after I attempt it.`
3. **Debug statistical code:**
   `Here is my t.test() code and the output: [PASTE CODE/OUTPUT]. Help me determine whether I used the correct test and arguments. Do not assume my interpretation is correct.`
4. **Critically evaluate an interpretation:**
   `Give me a plausible but incorrect interpretation of a t-test or ANOVA result. Ask me to identify the problem and explain why the interpretation is incorrect.`
5. **Connect to a career:**
   `How are comparisons of means used in [MY FIELD]? Give me three examples of real professional research questions involving means and explain which statistical method could address each question. Provide sources I can verify.`

# 7 Methods for the Proportion

So far, our statistical tests have focused on numerical outcomes such as age, education, and work hours. However, many research questions concern the proportion of times a certain outcome occurs in a population under study. For example, we can ask: *Is the proportion of respondents who believe men are more emotionally suited for politics than women less than 20%?*

## 7.1 One-Sample $z$-Test for Proportion

The GSS variable `fepol` records responses to the statement "Most men are better suited emotionally for politics than are most women." Before conducting a test, we should examine the observed counts and proportions.

```
gssdat_sub |>
  filter(!is.na(fepol)) |>
  count(fepol) |>
  mutate(proportion = n / sum(n))
```

```
##      fepol   n proportion
## 1    agree 164  0.1838565
## 2 disagree 728  0.8161435
```

The null hypothesis is that the population proportion who agree is 0.20. The alternative hypothesis is that this proportion is less than 0.20. We can conduct a one-sample $z$-test for the proportion using the `prop.test()` function. The argument `correct = FALSE` removes the continuity correction since the sample size is very large.

```r
prop.test(x = 164, n = 164+728, p = 0.20, alternative = "less",
          correct = FALSE, conf.level = 0.95)
```

```
##
##  1-sample proportions test without continuity correction
##
## data:  164 out of 164 + 728, null probability 0.2
## X-squared = 1.4529, df = 1, p-value = 0.114
## alternative hypothesis: true p is less than 0.2
## 95 percent confidence interval:
##  0.0000000 0.2061354
## sample estimates:
##         p
## 0.1838565
```

Among the respondents with valid responses, 18.39% agreed. The one-sample $z$-test for the proportion indicated that there is no evidence that the proportion of individuals who agree that men are more emotionally suited for politics is less than 0.20.

To obtain a two-sided confidence interval, we need to set `alternative = "two.sided"`.

```r
prop.test(x = 164, n = 164+728, p = 0.20, alternative = "two.sided",
          correct = FALSE, conf.level = 0.95)
```

```
##
##  1-sample proportions test without continuity correction
##
## data:  164 out of 164 + 728, null probability 0.2
## X-squared = 1.4529, df = 1, p-value = 0.2281
## alternative hypothesis: true p is not equal to 0.2
## 95 percent confidence interval:
##  0.1598098 0.2106145
## sample estimates:
##         p
## 0.1838565
```

With 95% confidence the proportion of individuals who agree that men are more emotionally suited for politics is between 0.160 and 0.211. This confidence interval includes 0.20 which agrees with the results of the hypothesis test.

## 7.2 Two-Sample $z$-Test for Difference in Proportions

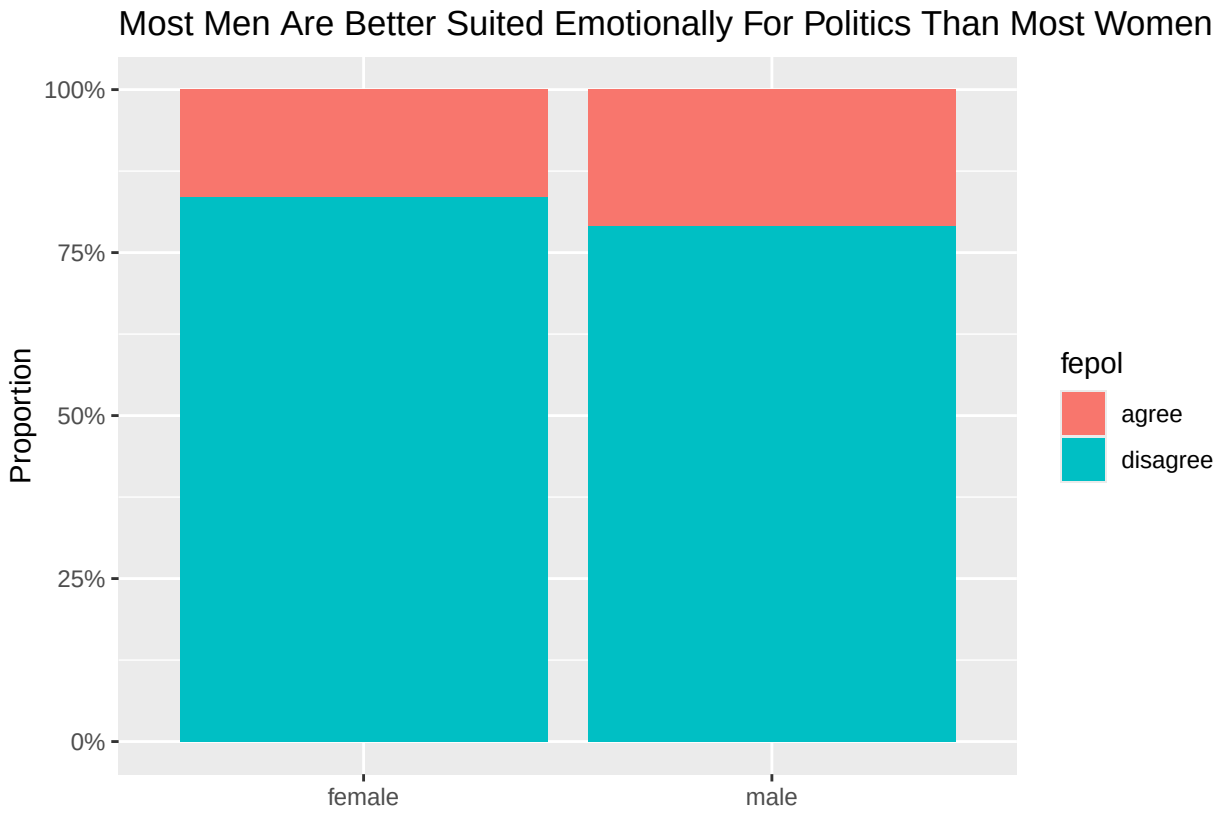
The one-sample proportion test examined one population proportion against a hypothesized value. We can extend the same reasoning to two groups. Suppose we want to ask: *Do men and women differ in the proportion who agree that men are more emotionally suited for politics than women?* The variables are `fepol` and `sex`.

Before conducting the test, we compare the observed proportions and obtain a bar chart to see the group differences directly.

```
gssdat_sub |>
  filter(!is.na(sex), !is.na(fepol)) |>
  group_by(sex) |>
  summarize(n = n(),
            count_agree = sum(fepol == "agree"),
            prop_agree = mean(fepol == "agree"))
```

```
## # A tibble: 2 x 4
##   sex        n count_agree prop_agree
##   <chr>  <int>       <int>      <dbl>
## 1 female   509          84      0.165
## 2 male     383          80      0.209
```

```
gssdat_sub |>
  filter(!is.na(sex), !is.na(fepol)) |>
  ggplot(aes(x = sex, fill = fepol)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Most Men Are Better Suited Emotionally For Politics Than Most Women",
       x = "", y = "Proportion")
```




Based on the sample data there does appear to be a higher rate of agreement among males than females, but a formal statistical test will help us determine if the difference is statistically significant.

The null hypothesis states that the population proportions are equal. The alternative hypothesis states that they differ. Like the one-sample case, we can use `prop.test()` to conduct this test by providing the counts and sample sizes to `prop.test()`. We must provide the count and sample size for each group in the `x` and `n` arguments.

```r
prop.test(x = c(84, 80),
          n = c(509, 383),
          alternative = "two.sided",
          correct = FALSE,
          conf.level = 0.95)
```

```
##
##  2-sample test for equality of proportions without continuity correction
##
## data:  c(84, 80) out of c(509, 383)
## X-squared = 2.8003, df = 1, p-value = 0.09425
## alternative hypothesis: two.sided
## 95 percent confidence interval:
##  -0.09578399  0.00808836
## sample estimates:
##    prop 1    prop 2
## 0.1650295 0.2088773
```

The output tests whether the two population proportions are equal and provides a confidence interval for their difference. Since we are analyzing the different in proportions, the order of subtraction is important here was the case for the two sample $t$-test. The order of the counts given for `x` and `n` determines the order of subtraction.

Based on the two-sample $z$-test for proportions there is no significant difference in the proportion of individuals who believe that most men are better suited emotionally for politics than are most women. We fail to reject the null hypothesis that the proportions differ. With 95% confidence 9.57% less to 0.81% more women agree with the statement than men. The upper bound of this interval is very close to zero, however, and the effect size is 0.21-0.17 = 0.04.

These results may be practically significant though they are not statistically significant. A statistically detectable difference does not necessarily mean that the difference is large or substantively important. Conversely, a visually noticeable difference in a small sample may not provide strong statistical evidence. This is why our workflow begins with a visualization and descriptive statistics.

This test also does not establish why the groups differ. Sex, attitudes, social experiences, age, education, and many other characteristics may be related. An observational comparison of two groups cannot by itself identify a causal explanation.

## 7.3 Sign Test for Median

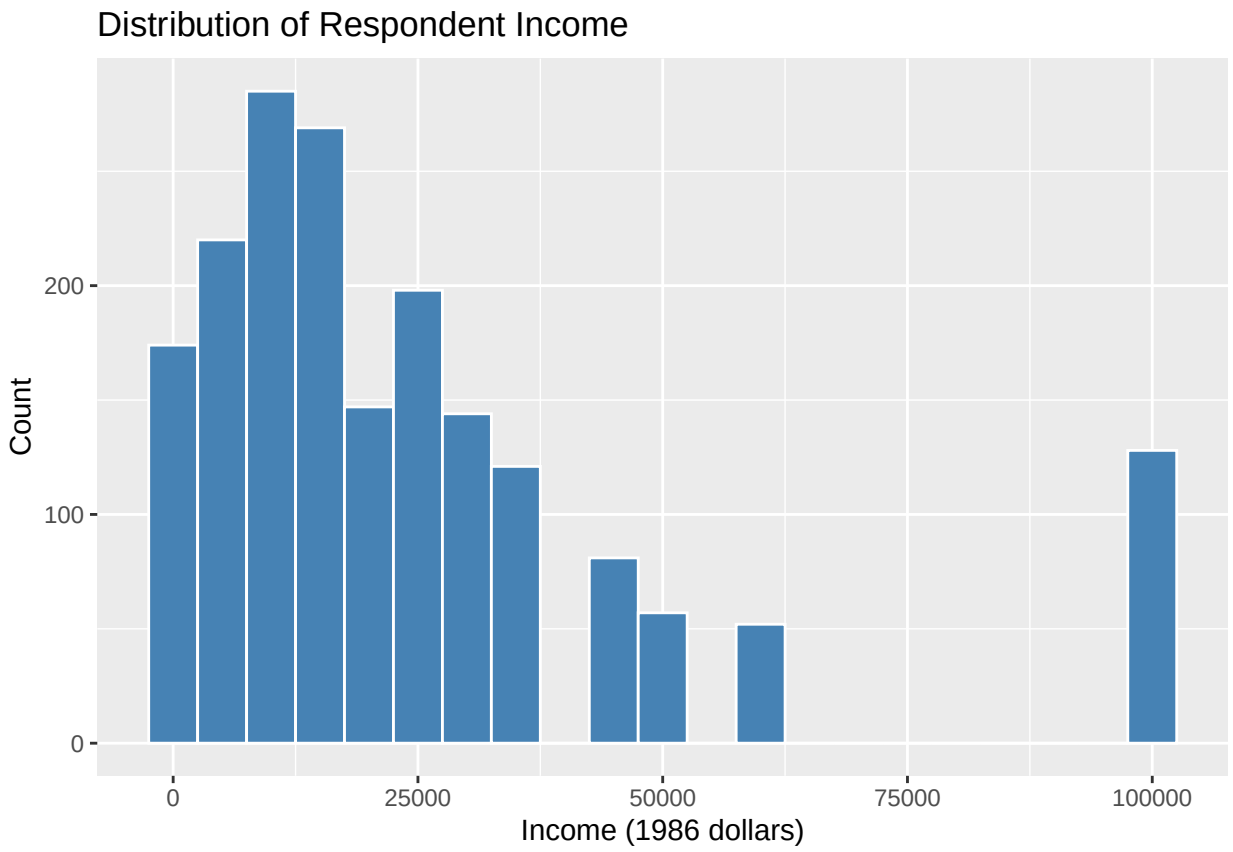
The methods we have used so far for numerical variables have focused primarily on means. The mean is useful, but it can be strongly affected by unusually large or small observations. Income is a particularly clear example. A small number of respondents with very high incomes can pull the mean upward, making the median a more useful description of the typical respondent. We will now ask the question *Is the median respondent income different from $20,000 in constant 1986 dollars?*

The variable `realrinc` measures respondent income in constant 1986 dollars. Before choosing a statistical method, we should inspect its distribution.

```r
income_sub <- gssdat_sub |>
  filter(!is.na(realrinc))

income_sub |>
```

```
  ggplot(aes(x = realrinc)) +
  geom_histogram(binwidth = 5000,
                 fill = "steelblue",
                 color = "white") +
  labs(title = "Distribution of Respondent Income",
       x = "Income (1986 dollars)",
       y = "Count")
```




We can calculate several descriptive statistics to see how strongly the mean and median differ.

```
income_sub |>
  summarize(n = n(),
            mean_income = mean(realrinc),
            median_income = median(realrinc),
            sd_income = sd(realrinc),
            min_income = min(realrinc),
            max_income = max(realrinc))
```

```
##      n mean_income median_income sd_income min_income max_income
## 1 1876    24892.37         16335  24271.74      181.5   97883.49
```

The distribution is not symmetric but rather right skewed and the mean is substantially greater than the median. The median provides a more representative description of the center in this case.

A sign test asks whether the population median is equal to a hypothesized value. We compare each observation with the hypothesized median and classify it as above, below, or exactly equal to that value.

Observations exactly equal to the hypothesized median are excluded. If the null hypothesis about the median is true, approximately half of the remaining observations should be above the hypothesized value, and half should be below it.

In this example, we use `prop.test()` to carry out the corresponding test on the proportion of observations above the hypothesized median. The statistical idea is the sign test while the proportion test provides the computational method.

The null hypothesis is that the median income is $20,000 and the alternative hypothesis is that the median is not equal to $20,000. For every respondent, we compare income with $20,000 to determine whether the observed value is above, below, or equal to this threshold.

```
income_sign <- income_sub |>
  mutate(comparison = case_when(
      realrinc > 20000 ~ "Above",
      realrinc < 20000 ~ "Below",
      realrinc == 20000 ~ "Equal"))

income_sign |>
  count(comparison)
```

```
##   comparison    n
## 1      Above  781
## 2      Below 1095
```

Observations exactly equal to the hypothesized median do not provide evidence in either direction, so they are excluded from the sign test.

```
income_sign_test <- income_sign |>
  filter(comparison != "Equal")

income_sign_test |>
  summarize(
    n = n(),
    above = sum(comparison == "Above"),
    below = sum(comparison == "Below"),
    proportion_above = mean(comparison == "Above"))
```

```
##      n above below proportion_above
## 1 1876   781  1095        0.4163113
```

If the null hypothesis is true, the probability that an observation is above the hypothesized median is approximately 0.50. We can therefore test whether the observed proportion above the hypothesized median differs from 0.50. A difference from 0.50 corresponds to evidence that the population median differs from the hypothesized value.

```
prop.test(x = 781, n = 1876, p = 0.50, alternative = "two.sided",
          correct = FALSE, conf.level = 0.95)
```

```
##
##  1-sample proportions test without continuity correction
##
## data:  781 out of 1876, null probability 0.5
```

```
## X-squared = 52.557, df = 1, p-value = 4.18e-13
## alternative hypothesis: true p is not equal to 0.5
## 95 percent confidence interval:
##  0.3941980 0.4387667
## sample estimates:
##         p
## 0.4163113
```

There is strong evidence that the proportion of incomes above $20,000 is different from 0.50. We reject the null hypothesis that it is equal to 0.50. With 95% confidence the proportion is between 0.39 and 0.44. Since substantially fewer than half of the observations are above $20,000, the test provides strong evidence that the population median differs from $20,000.

The sign test uses less information than a test based on the actual distances from the hypothesized value. The test does not tell us how far observations tend to be from $20,000. It only uses their direction relative to the hypothesized median. This makes the method particularly useful when the distribution is strongly skewed or contains extreme observations. An income of $21,000 and an income of $200,000 are both simply counted as being above $20,000. The advantage is that the method is less sensitive to extreme values and does not depend on the shape of the income distribution in the same way a mean-based test does.

## 7.4 Summary

In this chapter, we learned how to analyze questions about proportions and medians:

- A one-sample proportion test compares a population proportion with a hypothesized value.
- A two-sample proportion test compares proportions between two independent groups.
- A sign test uses the direction of observations relative to a hypothesized median.
- Confidence intervals help describe the uncertainty surrounding estimated proportions and differences.

As with the methods for means, the statistical procedure should follow from the research question and the type of variable being analyzed.

## 7.5 Career Readiness Connection

The skills in this chapter support several career readiness competencies:

- **Critical Thinking**: Determining how categorical responses should be defined and analyzed.
- **Communication**: Explaining proportions, uncertainty, and effect sizes in understandable terms.
- **Technology**: Using R to perform reproducible statistical analyses.
- **Equity and Inclusion**: Recognizing that decisions about how categories are combined can change the question being asked and whose experiences are represented.

### 7.5.1 Using AI to Study and Apply This Chapter

1. **Study method selection:**
   `Give me five research questions involving proportions or medians. Ask me to decide whether each requires a one-sample proportion test, two-sample proportion test, or sign test. Make the examples different enough that I cannot choose the method from the wording alone.`

2. **Practice R:**
   `Give me a categorical GSS-style variable and a research question involving a population proportion. Have me determine the null hypothesis, alternative hypothesis, sample proportion, R code, and interpretation.`
3. **Debug an analysis:**
   `Here is my prop.test() code and output: [PASTE CODE/OUTPUT]. Check whether the counts, sample sizes, alternative hypothesis, and interpretation match the research question.`
4. **Critically evaluate category choices:**
   `Give me an example where combining categories of a survey variable changes the research question. Explain what information is lost and what information becomes easier to analyze.`
5. **Connect to a career:**
   `How are population proportions or median comparisons used in [MY FIELD]? Give me three concrete examples from research or professional practice and identify reliable sources I can use to investigate them further.`

# 8 Regression Methods

The previous methods have focused on comparing groups or testing a single population quantity. We now turn to questions about relationships between two or more numerical variables and predictive models for the relationship.

## 8.1 Simple Linear Regression

Consider educational attainment and income. A reasonable question to ask is whether respondents with more education tend to have different incomes. We can investigate this relationship using `educ`, which measures years of educational attainment, and `realrinc`, which measures respondent income in constant 1986 dollars.

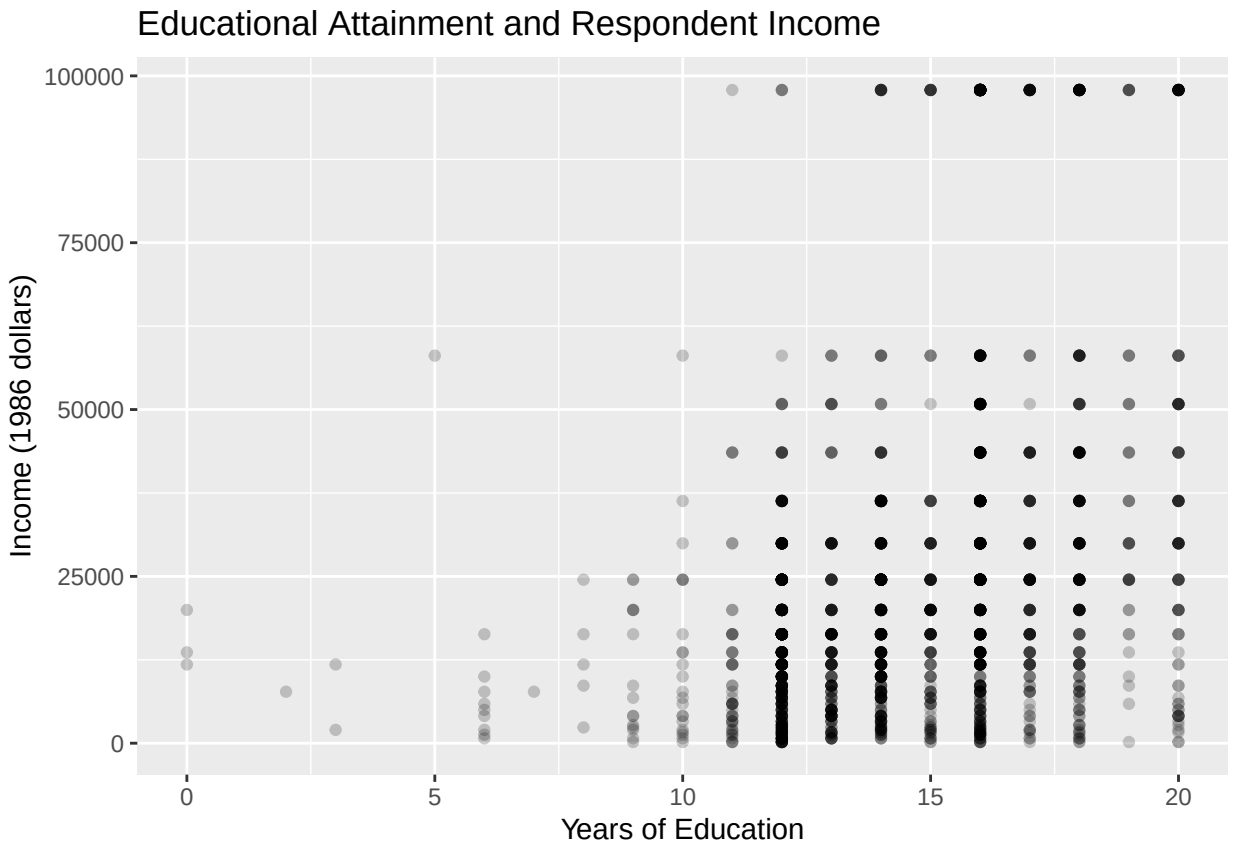
Our question is: *How is educational attainment associated with differences in respondents' personal income?* The word **associated** is important. These data are observational, so a regression model can describe and quantify a relationship without establishing that education causes income differences.

We begin, as always, with a visualization. A scatterplot is particularly useful here because regression assumes that we are examining a relationship that can reasonably be represented by a straight line. The plot also lets us see unusual observations and the substantial variability in income at similar levels of education.

The scatterplot and correlation coefficient indicate a moderate to weak linear relationship.

```
income_educ <- gssdat_sub |>
  select(educ, realrinc) |>
  filter(!is.na(educ), !is.na(realrinc))

income_educ |>
  ggplot(aes(x = educ, y = realrinc)) +
  geom_point(alpha = 0.2) +
  labs(title = "Educational Attainment and Respondent Income",
       x = "Years of Education",
       y = "Income (1986 dollars)")
```




```
cor(income_educ$realrinc, income_educ$educ)
```

```
## [1] 0.3534564
```

A simple linear regression models the expected value of the outcome as a linear function of one predictor. Here, `realrinc` is the outcome and `educ` is the predictor. The slope describes the expected change in the outcome associated with a one-unit increase in education.

We fit the model with `lm()`.

```
income_model <- lm(realrinc ~ educ, data = income_educ)

summary(income_model)
```

```
##
## Call:
## lm(formula = realrinc ~ educ, data = income_educ)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -41740 -14170  -4661   7163  84667
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -21866.3     2910.1  -7.514 8.85e-14 ***
## educ          3189.4      195.2  16.335  < 2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 22710 on 1869 degrees of freedom
## Multiple R-squared:  0.1249, Adjusted R-squared:  0.1245
## F-statistic: 266.8 on 1 and 1869 DF,  p-value: < 2.2e-16
```

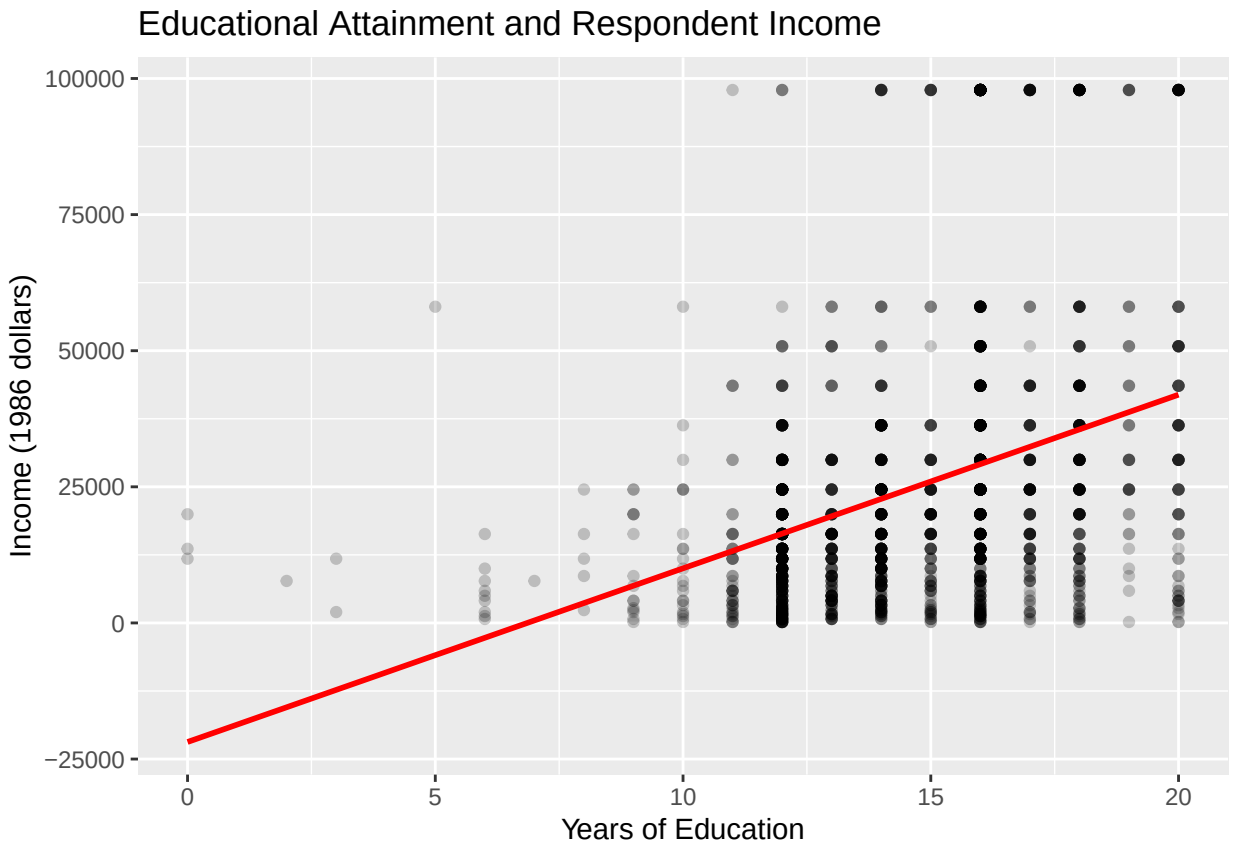
The coefficient table includes an intercept and a coefficient for `educ`. The intercept represents the predicted income when education equals zero. For an individual with no education the model predicts that their average income will be negative which is not sensible in this case. Although the intercept is mathematically necessary for the fitted line, it is not substantively meaningful because very few respondents have zero years of education.

The education coefficient is usually more directly relevant to the research question. It represents the predicted change in average income associated with one additional year of education. The model estimates that each additional year of education is associated with an average increase of $3,189 in predicted income.

### 8.1.1 Adding the Regression Line

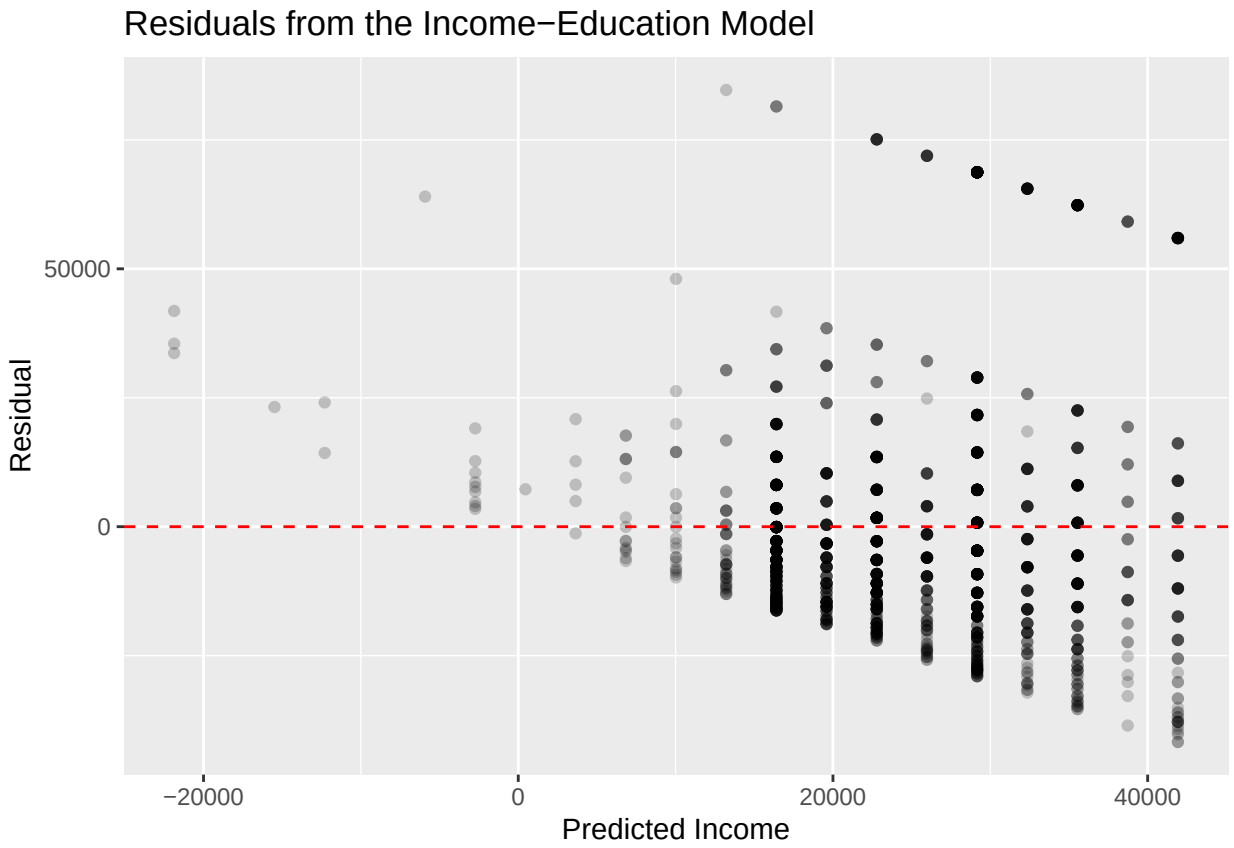
We can add the fitted regression line to the scatterplot to visualize model predictions for each value of the explanatory variable `educ`. The points show that substantial individual variation remains around the line.

```
income_educ |>
  ggplot(aes(x = educ, y = realrinc)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(title = "Educational Attainment and Respondent Income",
       x = "Years of Education",
       y = "Income (1986 dollars)")
```




We can also inspect the model's residuals. A residual is the observed outcome minus the model's predicted outcome. Visually, this is the distance between the regression line and each point. Residuals are prediction errors: they are the observed outcome minus the value predicted by the model. Examining their pattern helps us evaluate whether the model leaves systematic structure unexplained.

```
income_educ |>
  mutate(predicted_income = predict(income_model),
         residual = realrinc - predicted_income) |>
  ggplot(aes(x = predicted_income, y = residual)) +
  geom_point(alpha = 0.2) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Residuals from the Income-Education Model",
       x = "Predicted Income",
       y = "Residual")
```




Ideally, residuals should be scattered around zero without any strong pattern. Strong curvature, changing spread, or extreme observations can indicate that a simple linear model does not describe the data particularly well. In this case the spread of the residuals differs substantially as predicted income changes.

### 8.1.2 Interpreting $R^2$

The regression output also reports $R^2$. This describes the proportion of variation in the outcome accounted for by the linear relationship with the predictor in this sample. Only 12.5% of the variation in income is explained by education. This connects to the very weak correlation already observed.

A larger $R^2$ means that the model accounts for more of the observed variation. A small $R^2$ does not mean that the predictor is irrelevant; a variable can have a meaningful association with an outcome while much individual variation remains unexplained. Similarly, a large $R^2$ would not establish causation.

## 8.2 Multiple Linear Regression

Simple linear regression allows us to examine the relationship between one predictor and one numerical outcome. In some cases, the relationship may be more complex. Income, for example, may be related to education, age, and work hours at the same time.

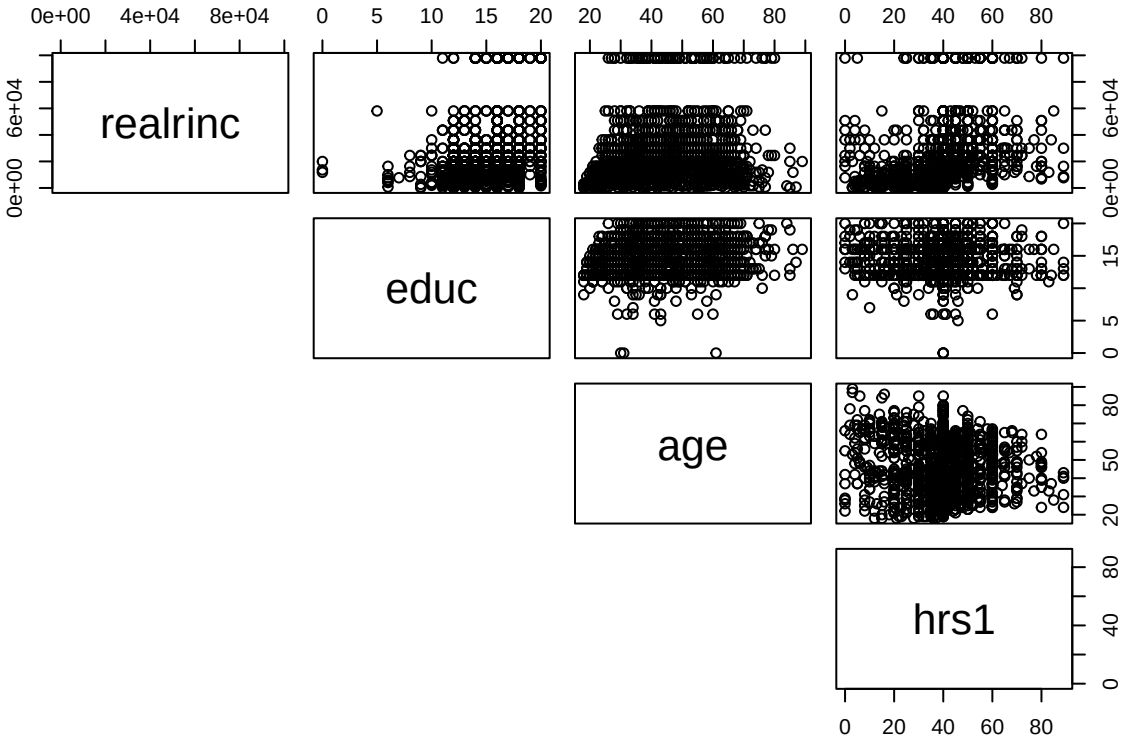
This leads to a more involved question: *How are respondents' incomes associated with educational attainment, age, and hours worked when these characteristics are considered simultaneously?*

The important phrase is *when considered simultaneously*. Multiple regression allows us to estimate the association between one predictor and the outcome while accounting for the other predictors included in the model. We will use `realrinc` as the outcome and `educ`, `age`, and `hrs1` as predictors.

```r
income_multiple <- gssdat_sub |>
  select(realrinc, educ, age, hrs1) |>
  filter(!is.na(realrinc),
         !is.na(educ),
         !is.na(age),
         !is.na(hrs1))
```

Before fitting the model, we should examine the relationship between the outcome and each predictor. We can use a scatterplot matrix to do this. The function `pairs()` creates scatterplots for each pair of variables involved in the analysis. The argument `lower.panel = NULL` removes the lower panel since it provides the same information.

```r
pairs(income_multiple, lower.panel = NULL)
```




This plot helps us to determine whether a linear model is a reasonable starting point and whether unusual observations or nonlinear patterns might affect the analysis. There does not appear to be a very strong association between `realrinc` and any of the predictors. In addition, there do not seem to be any relationships between predictors themselves.

We can also summarize the relationships using a correlation matrix which gives pairwise correlations. The matrix further confirms the weak relationships observed in the first row of the scatterplot matrix. The strongest relationship is between `realrinc` and `educ`. However, the predictors may explain more about `realrinc` in combination than individually.

```r
cor(income_multiple)
```

```
##             realrinc         educ          age         hrs1
## realrinc 1.0000000  0.353278774  0.11877766  0.281927883
## educ     0.3532788  1.000000000  0.09330603 -0.001150005
```

```
## age      0.1187777  0.093306030  1.00000000 -0.117107837
## hrs1     0.2819279 -0.001150005 -0.11710784  1.000000000
```

We can fit the model using the same `lm()` function used for simple regression. The difference is that we include several predictors on the right side of the formula using +.

```
income_multiple_model <- lm(realrinc ~ educ + age + hrs1, data = income_multiple)

summary(income_multiple_model)
```

```
##
## Call:
## lm(formula = realrinc ~ educ + age + hrs1, data = income_multiple)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -52195 -12417  -4379   6105  77702
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -49678.17    3967.58 -12.521  < 2e-16 ***
## educ          3126.98     212.60  14.708  < 2e-16 ***
## age            210.91      40.65   5.188 2.42e-07 ***
## hrs1           528.69      41.59  12.711  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 21500 on 1455 degrees of freedom
## Multiple R-squared:  0.219,  Adjusted R-squared:  0.2174
## F-statistic:   136 on 3 and 1455 DF,  p-value: < 2.2e-16
```

The coefficient for `educ` now has a more specific interpretation than it did in the simple regression model. It represents the predicted change in the average income associated with a one-year increase in education **while holding age and hours worked constant**.

Similarly, the coefficient for `age` represents the predicted change in income as age increases by one unit for respondents with the same education and hours worked. The coefficient for `hrs1` can be interpreted similarly.

The phrase “holding other variables constant” is central to interpreting multiple regression coefficients. All the model coefficients are positive indicating that these variables are associated with an increase in income. However, only about 22% of the variation in income is explained by these three variables based on the $R^2$.

### 8.2.1 Visualizing Model Predictions

Multiple regression is harder to visualize because the model contains several predictors simultaneously. We can nevertheless use predicted values to help understand the model.

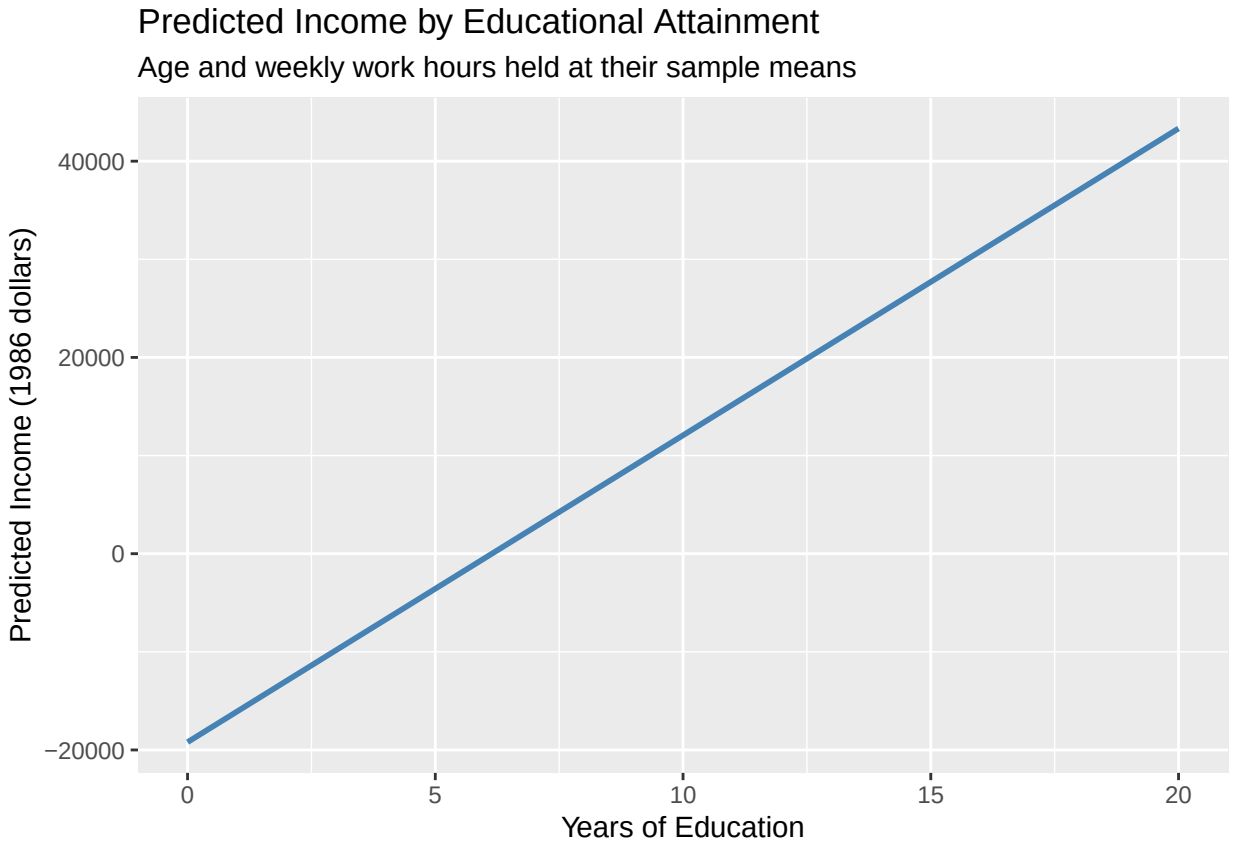
For example, we can examine predicted income across education while holding age and work hours at their sample means.

```r
prediction_data <- data.frame(educ = 0:20,
                              age = mean(income_multiple$age),
                              hrs1 = mean(income_multiple$hrs1))

prediction_data$predicted_income <- predict(income_multiple_model,
                                            newdata = prediction_data)
```

We can plot the predicted relationship.

```r
ggplot(prediction_data, aes(x = educ, y = predicted_income)) +
  geom_line(color = "steelblue", linewidth = 1) +
  labs(title = "Predicted Income by Educational Attainment",
       subtitle = "Age and weekly work hours held at their sample means",
       x = "Years of Education",
       y = "Predicted Income (1986 dollars)")
```




This graph shows the model's predicted mean income for different education levels under a particular set of values for the other predictors. Changing the values held constant would change the predictions. This illustrates an important feature of multiple regression: predictions depend on the combination of variables included in the model.

### 8.2.2 Examining Residuals

We should also examine whether the fitted model leaves systematic patterns in the residuals.

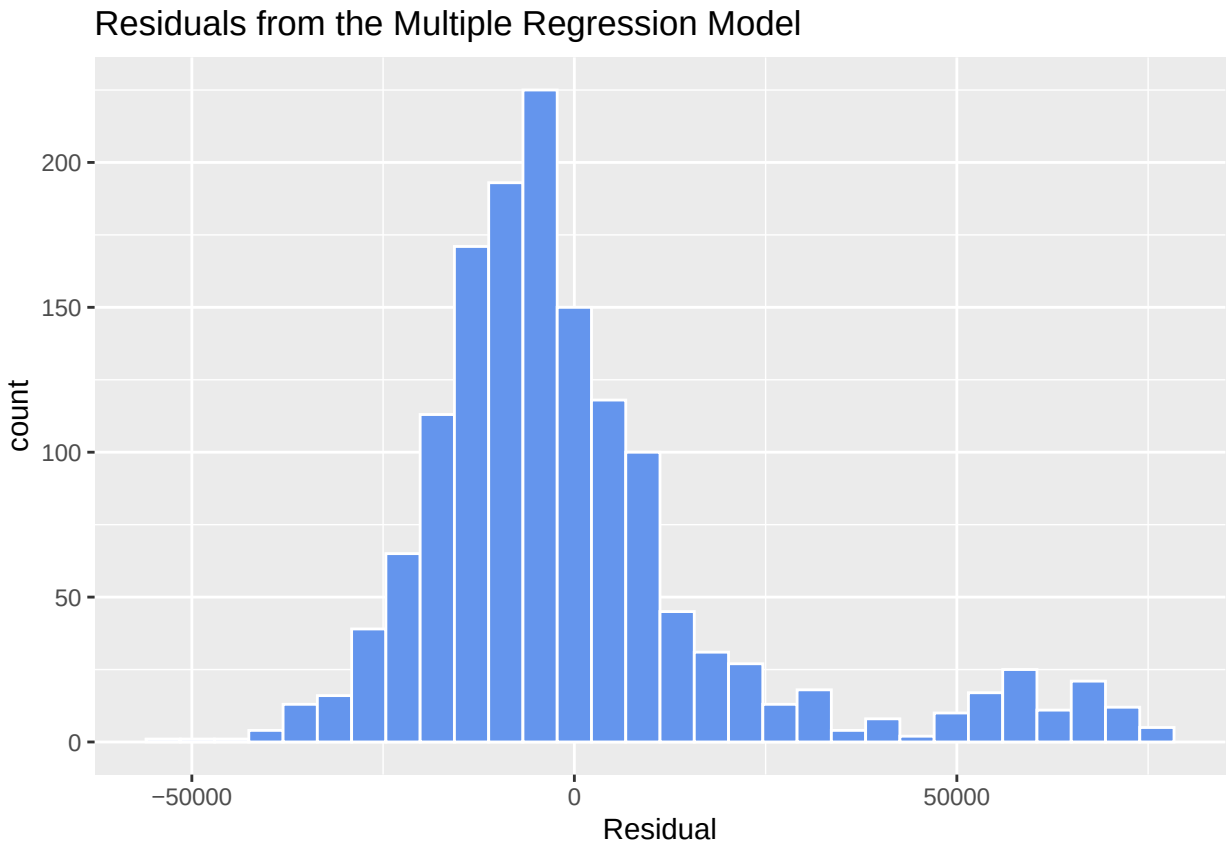
```
income_multiple |>
  mutate(predicted_income = predict(income_multiple_model),
         residual = realrinc - predicted_income) |>
  ggplot(aes(x = predicted_income, y = residual)) +
  geom_point(alpha = 0.2) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Residuals from the Multiple Regression Model",
       x = "Predicted Income",
       y = "Residual")
```




We look for systematic curvature, clusters, or a widening or narrowing spread in the residual plot since these patterns may indicate that the linear model does not fully capture the structure of the data. There are many values above and below the horizontal line at 0 which indicates that the model struggles with both under- and over-prediction of income. Moreover, the residuals exhibit non-constant variance.

Examining a histogram of the residuals we see that the distribution is right skewed rather than normal.

```
income_multiple |>
  mutate(predicted_income = predict(income_multiple_model),
         residual = realrinc - predicted_income) |>
  ggplot(aes(x = residual)) +
  geom_histogram(fill = "cornflowerblue", color = "white") +
  labs(title = "Residuals from the Multiple Regression Model",
       x = "Residual")
```




We should use caution when interpreting the regression results because the residual plots show non-constant variance and right skewness. These patterns indicate that some assumptions underlying the usual regression inference are not well satisfied. In addition, these are survey data, so the observations should not automatically be treated as independent observations from a simple random sample. These limitations do not prevent us from using the model to explore associations, but they should be considered when making inferential claims.

## 8.3 Summary

In this chapter, we learned how regression can be used to investigate relationships between numerical variables.

- Simple linear regression describes the association between one numerical predictor and a numerical outcome.
- A scatterplot helps us evaluate the relationship before fitting a model.
- The slope describes the predicted change in the outcome associated with a one-unit change in the predictor.
- $R^2$ describes the proportion of observed variation accounted for by the linear model.
- Residual plots help us evaluate whether the model leaves systematic patterns unexplained.
- Multiple linear regression allows several predictors to be considered simultaneously.
- Regression coefficients describe associations while holding the other predictors in the model constant.
- Regression with observational data does not by itself establish causation.

The goal is not simply to fit a model, but to determine whether the model is appropriate for the question and to communicate what the model can and cannot tell us.

## 8.4 Career Readiness Connection

The skills in this chapter support several career readiness competencies:

- **Critical Thinking**: Evaluating relationships, model assumptions, and limitations.
- **Technology**: Building, evaluating, and interpreting predictive models in R.
- **Communication**: Explaining regression coefficients and model results in context.
- **Professionalism**: Clearly distinguishing association, prediction, and causation.

Regression is widely used in professional settings, but responsible analysis requires recognizing that a model's output is not automatically evidence of a causal relationship.

### 8.4.1 Using AI to Study and Apply This Chapter

1. **Study regression concepts:**
   `Teach me the difference between correlation, simple linear regression, multiple linear regression, and causation. Then give me five examples and ask me to identify which concept each example illustrates.`
2. **Practice R:**
   `Give me a small dataset and a research question that could be investigated with simple linear regression. Have me choose the variables, create a scatterplot, fit the model, interpret the slope and R-squared, and examine residuals.`
3. **Debug regression code:**
   `Here is my lm() code and regression output: [PASTE CODE/OUTPUT]. Check whether I specified the outcome and predictors correctly and help me interpret the coefficients without making causal claims.`
4. **Critically evaluate a model:**
   `Give me a regression result with a misleading interpretation involving R-squared, residuals, or causation. Ask me to identify the problem before explaining it.`
5. **Connect to a career:**
   `Find three examples of regression being used in [MY FIELD]. For each example, identify the outcome, predictors, purpose of the model, and an important limitation. Provide sources I can verify.`

# 9 Practice Problems

These practice problems are an opportunity to apply the skills and methods you have learned throughout this manual. Use the codebook in Appendix A to review the variable names and definitions and refer to previous chapters whenever you need a reminder of R functions, statistical methods, or examples of code. You may also use AI as a learning tool: ask it to explain a concept, help you troubleshoot an error, or guide you toward a solution when you are stuck. You can also search the previous chapters for similar examples and adapt the code to the new question.

## 9.1 RStudio

1. Open the help file for the `mean` function with `?mean`.
2. Create a simple plot with `plot(1:5)`.
3. Change the RStudio theme using `Tools` → `Global Options` → `Appearance`

## 9.2 Tidyverse

4. Select only the variables needed to explore whether people's views on whether women are not suited for politics (`fepol`) have changed over time (`year`). Return a random sample of 10 rows.
5. How many respondents in 2018 reported that they are "very happy"? Keep only that `year` and happiness category (`happy`). Return the resulting dataset with only the variables used for filtering.
6. Create a variable `above_med_inc` for whether a respondent's `income` bracket is above or below the median income bracket in the dataset. Return a sample of 10 rows from the resulting dataset containing only `income` and `above_med_inc`.
7. What is the average number of children (`childs`) for each `marital` status?
8. Arrange `marital` status categories by average `age` (oldest to youngest).
9. Find all respondents whose job title (`occ10`) includes "nurse" (case-insensitive). Return a random sample of 10 rows from the resulting dataset that contains only `occ10`, `income`, and `age`.
10. Replace "telephone" with "phone" in the occupation variable (`occ10`). Return a random sample of 10 rows from the resulting dataset containing only `occ10` and `income` for occupations whose titles include "phone."
11. **Challenge** Define "young professionals" as respondents aged 25–40 whose occupation contains "manager," "teacher," or "engineer." How has the number of young professionals changed over time?

## 9.3 Cleaning

12. How many rows and columns does the dataset have? What type of variable is `degree`?
13. Create a new dataset that removes any rows with missing values for `income`. Return a random sample of 10 rows with the variables `income` and `age`
14. Use `pivot_longer()` to create a dataset with `happy` and `health` responses in a single column called `response_type`. Return a random sample of 10 rows of the resulting dataset with the two new variables.

15. In the `happy` variable, change “not too happy” to “unhappy” and return the counts for each category using this new level.

16. **Challenge:** Create a dataset with only respondents from 2010–2020, remove any rows with missing age, and then calculate the average age for each marital status.

## 9.4 Data Visualization and Summarization

17. Use `realrinc` (respondent’s inflation-adjusted income) to visualize the income distribution. Set `binwidth = 10000`, use a different fill and color, and add appropriate axis labels and a title. What shape does the income distribution take? Is it symmetric or skewed? Are there clusters or long tails that suggest income inequality?

18. Investigate whether confidence in the education system (`coneduc`) has changed over time. Choose an appropriate visualization and summary statistic for this question. Explain why you chose them, create the visualization, and describe any notable changes across survey years.

19. Investigate whether the distribution of confidence in Congress (`conlegis`) differs across racial groups and whether that pattern appears similar across sexes. Choose an appropriate visualization, facet it by sex, and explain what you learn from the graph.

20. Choose an appropriate visualization for investigating whether weekly work hours (`hrs1`) differ across marital-status groups (`marital`). Create the visualization and explain why it is appropriate for the question. Then calculate one numerical summary that helps you compare the groups. Does the summary support what you see in the graph?

21. Create a boxplot to compare education across different racial groups. Use years of education (`educ`) as the numerical variable and race (`race`) as the grouping variable. Add a descriptive title and axis labels to clarify the purpose of the visualization. If helpful for interpretation, you may also order the groups in a meaningful way (for example, by median education). Which racial group shows the highest median education level? Are the differences between groups large or small compared to the spread within each group?

## 9.5 Methods for the Mean

22. Using 2024 survey data and `maeduc` conduct a paired $t$-test to investigate whether respondents and their mothers differ in educational attainment. Use appropriate visualizations and summary statistics and interpret the results of the hypothesis test and confidence interval.

23. Investigate whether average weekly work hours (`hrs1`) differ between respondents with a bachelor’s degree and respondents with a high school degree using 2024 survey data. Use appropriate visualizations and summary statistics and interpret the results of the hypothesis test and confidence interval.

24. Investigate whether respondents who are working full-time differ in average age from respondents who are retired using 2024 survey data. Use appropriate visualizations and summary statistics and interpret the results of the hypothesis test and confidence interval.

25. Investigate whether the average number of children (`childs`) differs across education categories (`degree`) using 2024 survey data. Begin by choosing a visualization that will help you decide whether an overall comparison is appropriate. Then conduct the appropriate analysis and, if warranted, conduct pairwise comparisons. Explain what the results do and do not tell you.

26. Investigate whether average respondent income (`realrinc`) differs across marital-status categories (`marital`) using 2024 survey data. Begin by visualizing the distributions and calculating group summaries. Conduct an appropriate overall analysis and, if warranted, conduct pairwise comparisons. Because income is strongly skewed, explain one limitation of comparing means in this context.

## 9.6 Methods for the Proportion

27. Investigate whether more than half of respondents describe themselves as "very happy." Create a binary variable distinguishing "very happy" from all other valid responses. Choose an appropriate visualization, calculate the sample proportion, conduct the hypothesis test, and interpret the confidence interval.

28. Create a binary health measure that identifies respondents reporting "good" or "excellent" health versus all other valid responses. Test whether the population proportion differs from 0.75. Explain how combining the original categories changes the research question.

29. Investigate whether the proportion of respondents reporting good or excellent health differs by sex. Choose an appropriate visualization and statistical method. Report the difference in sample proportions, confidence interval, hypothesis-test result, and an appropriate measure of effect size.

30. Investigate whether the proportion of respondents who report being very happy differs between respondents with and without a bachelor's degree. Before conducting the test, write the null and alternative hypotheses in words and explain why a two-sample proportion test is appropriate.

31. Investigate whether the median age at first marriage (`agewed`) differs from 25 years. Begin by inspecting the distribution and comparing the mean and median. Explain why a sign test may be more appropriate than a mean-based test for this question. Then conduct the sign test and interpret the result.

32. Investigate whether the median number of children (`childs`) differs from 2. Use a visualization to examine the distribution, identify observations equal to the hypothesized median, and conduct the sign test. Explain how the observations equal to 2 are handled and why.

## 9.7 Regression Methods

33. Investigate how age (`age`) is associated with respondent income (`realrinc`). Create a scatterplot with line of best fit and calculate the correlation. Fit a simple linear regression predicting `realrinc` from `age`. Report the estimated intercept, slope, and $R^2$. Create a residual plot and describe whether you see evidence of nonlinearity or unequal variability. Explain what the slope and $R^2$ mean in context and why the model should not be interpreted as showing that age causes differences in income.

34. Investigate the relationship between hours worked last week (`hrs1`) and respondent income (`realrinc`). Create a scatterplot with line of best fit, calculate the correlation, and fit a simple linear regression predicting `realrinc` from `hrs1`. Report the estimated intercept, slope, and $R^2$. Create a residual plot and describe whether you see evidence of nonlinearity or unequal variability. Explain why hours worked and income might be associated without hours worked being the only explanation for income differences.

35. Investigate how weekly work hours (`hrs1`) are associated with age (`age`), education (`educ`), and marital status (`marital`). Begin by examining appropriate visualizations for the numerical and categorical predictors. Fit a multiple regression model predicting `hrs1` from `age`, `educ`, and `marital`. Because `marital` is categorical, allow R to create indicator variables for the marital-status

categories. Identify the reference category for marital status and interpret the age, education, and marital-status coefficients while holding the other predictors constant. Report the model's $R^2$ and examine a residual plot. Compare what you learned from the visualizations with what you learned from the regression model. Identify one important variable that is not included in the model and explain how its omission could affect interpretation.

# 10 Appendix A: Shorter GSS Dataset Codebook

| Variable | Label / Meaning | Type | Common Values / Categories |
|---|---|---|---|
| year | Survey year | Numeric | 1972–2024 |
| age | Respondent's age | Numeric | 18–89+ |
| degree | Highest degree earned | Ordinal | less than high school, high school, associate/junior college, bachelor's, graduate |
| income | Respondent's income bracket | Ordinal | Numeric codes representing income brackets |
| occ10 | Occupation code (2010 scheme) | Categorical | 4-digit census occupation codes |
| incomecat | Respondent's income category | Ordinal | Labeled income categories |
| trustfam | Trust in family | Ordinal | not at all, only some, a lot |
| trustdoc | Trust in doctors | Ordinal | not at all, only some, a lot |
| wrkstat | Labor force status | Categorical | working full-time, part-time, unemployed, retired |
| id | Respondent identifier | Numeric | Unique ID |
| marital | Marital status | Categorical | married, widowed, divorced, separated, never married |
| agewed | Age when first married | Numeric | 15–80+ |
| conjudge | Confidence in courts/judges | Ordinal | not at all, only some, a lot |
| realrinc | Respondent's income (1986 constant dollars) | Numeric | Continuous |
| hrs1 | Hours worked last week | Numeric | 0–80+ |
| conpress | Confidence in press | Ordinal | not at all, only some, a lot |
| race | Race of respondent | Categorical | White, Black, Other |
| consci | Confidence in science | Ordinal | not at all, only some, a lot |
| fepol | Attitude: women suited for politics | Nominal | agree, disagree |
| happy | General happiness | Ordinal | not too happy, pretty happy, very happy |
| childs | Number of children | Numeric | 0–8+ |
| health | Self-rated health | Ordinal | poor, fair, good, excellent |
| realinc | Family income (1986 constant dollars) | Numeric | Continuous |
| coneduc | Confidence in education system | Ordinal | not at all, only some, a lot |
| conlegis | Confidence in Congress | Ordinal | not at all, only some, a lot |
| sex | Respondent's sex | Categorical | male, female |
| educ | Respondent's education | Numeric | 0–20 years |
| paeduc | Father's education | Numeric | 0–20 years |
| relig | Respondent's religious preference | Categorical | See GSS codebook for response categories |
| res16 | Type of place lived in when 16 years old | Categorical | suburb, large city, small city/town, medium-size city, open country/farm |

# 11 Appendix B: Transitioning from Python to R Using AI

AI is integrated throughout the earlier chapters of this manual. Each chapter includes prompts that can be used to study concepts, practice R, troubleshoot problems, evaluate AI-generated responses, and investigate how statistical skills are used in different careers. Those activities provide short, task-specific opportunities to use AI while learning the material.

This appendix goes further by focusing specifically on using AI to translate between R and Python, improve prompts, verify generated code, debug errors, and develop habits for independent learning.

## 11.1 Why Use AI to Bridge R and Python?

Users of this manual may be learning programming for the first time, while others may already have experience in Python. Because this lab manual is written in `R`, those with prior Python experience may find themselves mentally "translating" code as they work. This appendix introduces a structured way to support that process using AI.

### 11.1.1 R and Python: Different Syntax, Similar Ideas

`R` and Python are two of the most widely used programming languages in data analysis. While they differ in syntax and style, they rely on many of the same core ideas:

- Importing and inspecting data
- Selecting and filtering observations
- Creating new variables
- Summarizing information
- Visualizing results

For example, the following `R` code uses the `tidyverse` to filter and summarize data:

```
gssdat |>
  filter(year == 2022) |>
  summarize(avg_age = mean(age, na.rm = TRUE))
```

A Python user might expect to perform the same task using the `pandas` library. Although the syntax looks different, the logic is the same:

- Start with a dataset
- Filter rows
- Compute a summary statistic

Learning to recognize these shared ideas is more important than memorizing exact syntax.

### 11.1.2 Why People May Prefer Python

Python is widely used across industries such as technology, finance, and data science. Some readers may already be familiar with tools like:

- `pandas` for working with data
- `NumPy` for numerical operations

Because of this familiarity, translating `R` code into Python can make new concepts feel more accessible. However, it is important to recognize that `R`—especially through the tidyverse—offers a powerful and expressive system for data analysis that is widely used in research and the social sciences.

Rather than choosing one language over the other, this chapter encourages readers to build connections between them.

### 11.1.3 The Role of AI in Learning Code

AI tools such as ChatGPT and Copilot can help translate code between languages, explain unfamiliar syntax, and suggest alternative approaches.

However, AI is most useful when used actively, not passively.

- Passive use: copying AI-generated code without understanding it
- Active use: asking AI to explain, compare, and justify code

In this appendix, readers will learn how to:

- Write prompts that guide AI effectively
- Ask for explanations alongside code
- Check whether AI outputs are correct
- Use AI to strengthen their understanding of data analysis concepts

Think of AI as a tutor or collaborator, not a shortcut.

### 11.1.4 A Simple Example of AI-Assisted Translation

Suppose you encounter the following R code:

```
gssdat |>
  select(age, income) |>
  filter(age > 30)
```

Instead of manually rewriting this in Python, you could ask an AI tool:

```
Translate this R tidyverse code into Python using pandas and explain each step.
```

The key is not just the translation—it is the explanation that helps you understand how the two languages relate.

Later in this chapter, you will learn how to write prompts like this effectively and refine them when needed.

### 11.1.5 Academic Integrity and Responsible Use

AI is a powerful tool, but it must be used responsibly:

- Try to understand any code submitted
- Use AI to support learning, not replace it
- Always review and test AI-generated code

Using AI effectively is a professional skill. In many careers, knowing how to ask the right questions and evaluate automated outputs is just as important as writing code from scratch.

### 11.1.6 Key Takeaway

R and Python may look different, but they are built on the same analytical foundations. AI can help bridge the gap between them—but only if you use it thoughtfully.

In the next section, you will learn how to write effective prompts that allow AI to become a powerful learning tool rather than just a code generator.

## 11.2 How to Prompt AI for Code Translation

Effectively using AI to translate code from R to Python depends less on the tool itself and more on how the prompt is written. Clear, detailed prompts help AI generate more accurate, useful, and interpretable outputs. This section provides a framework for developing strong prompting habits.

### 11.2.1 The Anatomy of an Effective Prompt

An effective prompt for translating R code into Python typically includes three key elements:

1. Specify the task clearly > Indicate that the goal is to translate code from R to Python.
2. Provide context > Include information about the dataset, variables, or analytical goal.
3. Request explanation > Ask the AI to explain what the translated code is doing.

A general template is shown below:

```
Translate the following R (tidyverse) code into Python using pandas.
Explain each step of the Python code and how it corresponds to the R code.

[R CODE HERE]
```

This structure encourages AI to act as both a translator and a teacher.

### 11.2.2 Example: Weak vs. Strong Prompts

The quality of AI output is highly sensitive to the quality of the prompt. Consider the following comparison:

**Weak Prompt**

```
Convert this to Python:


gssdat |> filter(year == 2022)
```

**Issues:**

- Does not specify which Python library to use
- Provides no explanation request
- Lacks context about the dataset or goal

**Strong Prompt**

```
Translate the following R tidyverse code into Python using pandas.
The dataset is called gssdat and contains survey data.
Explain how the Python code filters the data and how it compares to the R code.

gssdat |> filter(year == 2022)
```

**Why this works better:**

- Specifies tidyverse → pandas translation
- Provides dataset context
- Requests conceptual explanation
- Encourages understanding, not just output

### 11.2.3 Adding Context Improves Results

AI performs better when it understands the purpose of the code. For example:

```
Translate the following R code into Python using pandas.
This code selects age and income variables and filters respondents over age 30.
Explain each step and describe the output.

gssdat |>
  select(age, income) |>
  filter(age > 30)
```

Including the purpose (“filters respondents over age 30”) helps the AI produce clearer and more accurate explanations.

### 11.2.4 Asking for Explanation, Not Just Code

One of the most valuable uses of AI is its ability to explain relationships between languages. This can be done using prompts that include phrases such as:

```
Explain each step

Compare this to the R version

Describe what the output will look like
```

This shifts the focus from code generation to conceptual understanding, which is especially important when learning how different programming languages implement similar ideas.

### 11.2.5 Iterative Prompting: Refining the Output

AI prompting is often an iterative process. The first response may not be perfect, and follow-up prompts can improve clarity and accuracy.

**Step 1: Initial Prompt**

```
Translate this R code into Python using pandas:

gssdat |> group_by(degree) |> summarize(avg_age = mean(age, na.rm = TRUE))
```

**Step 2: Follow-Up Prompt**

```
Can you rewrite the Python code using method chaining?
Also explain how missing values are handled in Python compared to R.
```

**Step 3: Clarification Prompt**

```
What does the groupby function return before aggregation?
```

This step-by-step refinement helps build deeper understanding and mirrors how analysts troubleshoot and learn in real-world settings.

### 11.2.6 Common Mistakes When Using AI

- Vague prompts

  (“Convert this”) without specifying tools or goals

- No request for explanation

  Leads to surface-level learning

- Ignoring errors in output

AI-generated code may contain mistakes or assumptions

- Overly complex inputs at the start

  Large code blocks can produce confusing responses; start small

- Assuming one-to-one translation

  Not all R functions map directly to Python equivalents

Recognizing these issues can help you develop more effective habits when working with AI.

### 11.2.7 Mini Activity: Improving a Prompt

**Weak Prompt**

```
Turn this into Python:

gssdat |> mutate(young = age < 30)
```

**Reflection Questions**

- What information could make this prompt more effective?
- How might different prompts produce different outputs?
- What additional clarification could be added if the output is unclear?

Revise this prompt using your reflections. When you are done see how your answer compares to the example answer below.

**Example Improved Prompt**

```
Translate the following R tidyverse code into Python using pandas.
The dataset gssdat contains survey data.
This code creates a new variable indicating whether a respondent is under age 30.
Explain how the new variable is created in Python.

gssdat |> mutate(young = age < 30)
```

### 11.2.8 Key Takeaway

Prompting is a skill that improves with practice. Clear, specific, and context-rich prompts allow AI to function as a powerful tool for translating and understanding code. By emphasizing explanation, iteration, and critical evaluation, instructors can help learners use AI to build transferable programming knowledge across R and Python.

## 11.3 Translating Core Data Tasks

One of the most effective ways to move between R and Python is to focus on **concepts rather than syntax**. While the code may look different across languages, the underlying tasks—importing data, filtering rows, creating variables, and summarizing results—remain the same.

AI can assist in this process, but its greatest value lies in helping users recognize **how ideas transfer**, not just producing equivalent code. This section demonstrates how to use AI to translate core data tasks while maintaining a focus on conceptual understanding.

### 11.3.1 Data Import and Inspection

The first step in any data analysis workflow is importing and inspecting data.

**R Example**

```r
gssdat <- read.csv("files/gssdat.csv")

dim(gssdat)
names(gssdat)
summary(gssdat$age)
```

**AI Prompt Example**

```
Translate the following R code into Python using pandas.
The dataset is a CSV file called gssdat.csv.
Explain how the Python code imports the data and inspects its structure.

gssdat <- read.csv("files/gssdat.csv")

dim(gssdat)
names(gssdat)
summary(gssdat$age)
```

**Resulting Python Code (Typical Output)**

```python
import pandas as pd

gssdat = pd.read_csv("files/gssdat.csv")

gssdat.shape
gssdat.columns
gssdat["age"].describe()
```

| Task | R | Python |
|---|---|---|
| Import data | `read.csv()` | `pd.read_csv()` |
| Dimensions | `dim()` | `.shape` |
| Column names | `names()` | `.columns` |
| Summary statistics | `summary()` | `.describe()` |

**Key Insight**

Although the syntax differs, both languages:

- Load a dataset into memory
- Provide tools to inspect structure
- Allow quick summaries of variables

AI is most useful when asked to explain these parallels, helping learners see that the workflow—not the syntax—is what transfers across languages.

### 11.3.2 Data Wrangling Concepts

Data wrangling is central to this lab manual. The `tidyverse` provides a consistent grammar for these tasks, while Python commonly uses the `pandas` library.

Rather than memorizing translations, it is helpful to map core ideas.

| Task | R (tidyverse) | Python (pandas) |
|---|---|---|
| Select columns | `select()` | `df[["col1", "col2"]]` |
| Filter rows | `filter()` | `df[df["col"] == value]` |
| Create variables | `mutate()` | `df["new"] = ...` |
| Group + summarize | `group_by()` + `summarise()` | `groupby().agg()` |

#### Example: Filtering and Selecting

**R Code**

```
gssdat |>
  select(year, age, income) |>
  filter(year == 2022)
```

**AI Prompt**

```
Translate the following R tidyverse code into Python using pandas.
This code selects the variables year, age, and income, then filters the data to only include rows from 2
Explain how each step works in Python.

gssdat |>
  select(year, age, income) |>
  filter(year == 2022)
```

**Python Output (Typical)**

```
gssdat[["year", "age", "income"]][gssdat["year"] == 2022]
```

#### Example: Creating a New Variable

**R Code**

```
gssdat |>
  mutate(young_adult = age >= 18 & age <= 30)
```

**Python Output (Typical)**

```
gssdat["young_adult"] = (gssdat["age"] >= 18) & (gssdat["age"] <= 30)
```

**Example: Grouping and Summarizing**

**R Code**

```
gssdat |>
  group_by(degree) |>
  summarise(avg_age = mean(age, na.rm = TRUE))
```

**Python Output (Typical)**

```
gssdat.groupby("degree")["age"].mean()
```

**Reflection: What Stayed the Same?**

Across all examples:

- A dataset is the starting point
- Operations are applied step-by-step
- The goal is to transform or summarize data

What changes is **how the steps are written**, not **what the steps are doing**.

Encouraging reflection on these similarities helps build transferable skills. AI can support this by explaining *why* a translation works, not just *what* the translated code is.

### 11.3.3 Pipes vs. Method Chaining

One of the most noticeable differences between R and Python is how multiple operations are linked together.

**R: Pipe Operator**

```
gssdat |>
  filter(year == 2022) |>
  group_by(degree) |>
  summarise(avg_age = mean(age, na.rm = TRUE))
```

The pipe (|>) passes the result of one step into the next, creating a readable sequence of operations.

**Python: Method Chaining**

In Python, similar workflows are often written using method chaining:

```
gssdat[gssdat["year"] == 2022] \
    .groupby("degree")["age"] \
    .mean()
```

**AI Prompt to Highlight Structure**

```
Translate the following R code into Python using pandas method chaining.
Explain how each step in the chain corresponds to the pipe operator in R.

gssdat |>
  filter(year == 2022) |>
  group_by(degree) |>
  summarise(avg_age = mean(age, na.rm = TRUE))
```

#### 11.3.4 How AI Supports Understanding

When prompted effectively, AI can:

- Break down each step in the pipeline
- Show how intermediate results flow through the code
- Compare readability and structure across languages

This reinforces a key idea: both R and Python support sequential data transformations, even if they express them differently.

#### 11.3.5 Key Takeaway

Translating between R and Python is most effective when approached as concept mapping rather than direct substitution. AI can assist in this process by making relationships between tasks explicit, helping users recognize that:

- Data workflows are consistent across languages
- Syntax differences are secondary to conceptual understanding
- Clear prompts lead to clearer explanations and better learning outcomes

In the next section, this approach will be extended to data visualization, where stylistic differences between languages become more pronounced but the underlying goals remain the same.

### 11.4 Translating Visualizations

Data visualization is one of the areas where differences between R and Python are most visible. While both languages can produce high-quality graphics, they follow different design philosophies.

In R, visualizations are commonly created using the **grammar of graphics** implemented in `ggplot2`. In Python, visualizations are often created using libraries such as `matplotlib` and `seaborn`.

AI can help translate between these systems, but it is especially useful for explaining how the same visual idea is expressed differently across languages.

### 11.4.1 Conceptual Differences: Grammar vs Layers

**R** (`ggplot2`) builds plots layer-by-layer using a consistent grammar:

- Data
- Aesthetic mappings (`aes`)
- Geometric objects (`geom_*`)
- Labels and themes

**Python** (`matplotlib`/`seaborn`) is often more flexible but less standardized:

- Plots are created through function calls or method chaining
- Styling and customization may require additional steps

Despite these differences, both approaches aim to:

- Represent data visually
- Highlight patterns and relationships
- Communicate findings clearly

### 11.4.2 Translating Histograms

#### R Code

```r
gssdat |>
  ggplot(aes(x = age)) +
  geom_histogram(binwidth = 5, fill = "steelblue", color = "white") +
  labs(title = "Age Distribution",
       x = "Age",
       y = "Count")
```

#### AI Prompt Example

```
Translate the following R ggplot2 code into Python using seaborn or matplotlib.
Explain how the histogram is constructed and how the binwidth argument is handled in Python.

gssdat |>
  ggplot(aes(x = age)) +
  geom_histogram(binwidth = 5, fill = "steelblue", color = "white") +
  labs(title = "Age Distribution",
      x = "Age",
      y = "Count")
```

#### Python Output (Typical)

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=gssdat, x="age", binwidth=5, color="steelblue", edgecolor="white")

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```

#### Comparison

- Both specify the variable (`age`)
- Both control bin size (`binwidth`)
- Both customize labels and appearance

The main difference is structural:

- R layers components with +
- Python uses sequential function calls

### 11.4.3 Translating Scatterplots

#### R Code

```
gssdat |>
  ggplot(aes(x = age, y = realrinc)) +
  geom_point(alpha = 0.3) +
  labs(title = "Age and Income",
       x = "Age",
       y = "Income")
```

#### AI Prompt Example

```
Convert this ggplot2 scatterplot into Python using seaborn.
Explain how transparency (alpha) is handled and how the axes are defined.

gssdat |>
  ggplot(aes(x = age, y = realrinc)) +
  geom_point(alpha = 0.3)
```

#### Python Output (Typical)

```
sns.scatterplot(data=gssdat, x="age", y="realrinc", alpha=0.3)

plt.title("Age and Income")
plt.xlabel("Age")
plt.ylabel("Income")
plt.show()
```

### Comparison

- The mapping of variables (`x`, `y`) is conceptually identical
- Transparency (`alpha`) exists in both languages
- Labels are added separately in Python

This highlights a key insight: the visual relationship stays the same even when syntax changes.

### 11.4.4 Translating Bar Charts

#### R Code

```
gssdat |>
  ggplot(aes(x = race, fill = conpress)) +
  geom_bar(position = "fill") +
  labs(title = "Confidence in the Press by Race",
       x = "Race",
       y = "Proportion")
```

#### AI Prompt Example

```
Translate this stacked bar chart from ggplot2 into Python using seaborn.
Explain how the "fill" position (proportions) is represented in Python.

gssdat |>
  ggplot(aes(x = race, fill = conpress)) +
  geom_bar(position = "fill")
```

#### Python Output (Typical)

```
import pandas as pd

prop_data = (
    gssdat
    .groupby(["race", "conpress"])
    .size()
    .groupby(level=0)
    .apply(lambda x: x / x.sum())
    .reset_index(name="proportion")
)

sns.barplot(data=prop_data, x="race", y="proportion", hue="conpress")

plt.title("Confidence in the Press by Race")
plt.xlabel("Race")
plt.ylabel("Proportion")
plt.show()
```

#### Comparison

- R handles proportions automatically with `position = "fill"`
- Python often requires manual calculation of proportions
- Both ultimately display the same conceptual relationship

This example illustrates that translation is not always one-to-one. AI can help explain these differences and suggest appropriate workflows.

### 11.4.5 Stylistic vs Conceptual Differences

When translating visualizations, it is helpful to distinguish between:

**Conceptual Elements (Shared)**

- What variables are being plotted
- What relationship is being shown
- What type of graph is used

**Stylistic Elements (Different)**

- Default colors and themes
- How layers are added
- How labels and legends are formatted

AI can be especially useful for:

- Identifying which parts of a plot are conceptual vs stylistic
- Explaining why a translation may require additional steps
- Suggesting improvements for clarity or aesthetics

### 11.4.6 Using AI to Compare Outputs

A useful prompting strategy is to ask AI to compare plots directly:

```
Compare the ggplot2 and seaborn versions of this visualization.
What are the main differences in structure, defaults, and flexibility?
```

This encourages deeper analysis and helps users move beyond surface-level translation.

### 11.4.7 Key Takeaway

Visualizations in R and Python may look different in code, but they are built on the same goals: representing data clearly and effectively. AI can support translation by:

- Converting syntax between libraries
- Explaining structural differences
- Highlighting conceptual similarities

By focusing on *what the visualization is doing*, rather than just *how it is written*, users can develop flexible skills that transfer across tools, languages, and analytical contexts.

## 11.5 Becoming an Independent Learner with AI

AI can be a powerful support tool for translating code and learning new programming languages. However, its long-term value depends on how it is used. The goal is not simply to generate correct code, but to develop the ability to **evaluate, refine, and learn from AI-generated outputs**.

This section focuses on building habits that support independent learning across R, Python, and other analytical tools.

### 11.5.1 Verifying AI-Generated Code

AI-generated code should always be treated as a **first draft**, not a final answer. Verification is an essential step in any workflow.

Some practical strategies include:

- **Run the code and inspect the output**

  Does the result match expectations? Are there unexpected values or missing results?

- **Check intermediate steps**

  Break the code into smaller parts and examine what each step produces.

- **Compare with known results**

  If the same task has already been completed in R, compare outputs between R and Python.

- **Look for warnings or silent errors**

  Some issues (such as missing values) may not produce explicit errors but can still affect results.

AI can also assist in verification when prompted carefully:

```
This Python code was generated from R.
Can you check if it correctly replicates the logic and identify any potential issues?
```

This encourages AI to act as a reviewer rather than just a generator.

### 11.5.2 Debugging Strategies

Errors are a natural and valuable part of the learning process. AI can support debugging when used interactively.

**Ask AI to Explain Errors**

Instead of asking for a corrected version immediately, it is often more helpful to ask for an explanation:

```
I received this error in Python.
Can you explain what it means and what might be causing it?

[ERROR MESSAGE HERE]
```

Understanding the cause of an error builds transferable problem-solving skills.

**Compare Outputs Between R and Python**

When translating code, differences in results can reveal important insights.

For example:

- Are missing values handled the same way?
- Are group summaries calculated identically?
- Are categorical variables treated consistently?

A useful prompt might be:

```
The R and Python versions of this analysis produce different results.
What are possible reasons for the discrepancy?
```

This encourages deeper thinking about how each language processes data.

### 11.5.3 When Not to Rely on AI

AI is most useful when supporting learning, but there are situations where reliance should be limited:

- When foundational concepts are unclear

  It is often more effective to revisit basic ideas before translating code

- When outputs are accepted without question

  AI can produce incorrect or incomplete code

- When tasks become overly automated

  Over-reliance can reduce opportunities to practice problem-solving

- When documentation provides clearer answers

  Official resources are often more precise for specific functions or syntax

Recognizing these situations helps maintain a balance between efficiency and understanding.

### 11.5.4 Building Long-Term Skills

AI should be integrated into a broader learning strategy rather than used in isolation.

**Reading Documentation**

Both R and Python have extensive documentation that explains functions, arguments, and examples in detail. Learning to navigate documentation helps users:

- Verify AI-generated suggestions
- Explore alternative approaches
- Build confidence in independent problem-solving

AI can assist here as well:

```
Can you summarize the key points of the pandas groupby documentation and explain when to use it?
```

### 11.5.5 Using AI as a Tutor, Not a Shortcut

AI is most effective when treated as a learning partner. Productive uses include:

- Asking for step-by-step explanations
- Requesting comparisons between methods
- Exploring “what if” scenarios
- Breaking down complex code into smaller pieces

Less productive use involves copying code without engaging with the underlying ideas.

### 11.5.6 Key Takeaway

AI can accelerate learning and support transitions between programming languages, but its effectiveness depends on active engagement. By verifying outputs, debugging thoughtfully, and using AI as a guide rather than a replacement, learners can build flexible, durable skills that extend beyond any single tool or language.

## 11.6 Conclusion

The ability to work effectively with AI is increasingly relevant across many fields. This final section encourages the reader to reflect on how learning to use AI as part of the data analysis workflow could help prepare them for their career.

Think about each of the following questions and discuss your answers with a fellow learner:

- How does using AI change the way problems are approached in data analysis?
- What strategies help ensure that AI-generated results are accurate and reliable?
- In what situations might human judgment be more important than automated output?
- How can AI be used to support collaboration and communication in professional settings?
- What skills remain essential, even when AI tools are available?